

CS336 作业 1（基础）：构建 Transformer 语言模型

版本 26.0.3

CS336 课程组

2026 年春季

译者说明：本文是所附英文作业说明的中文翻译。函数名、变量名、类型标注、命令、数学符号和引用编号均尽量保留原样；如中英文表述存在歧义，请以课程发布的英文原文为准。本译文只翻译题目，不提供题目答案或实现代码。

1 作业概览

在本次作业中，你将从零开始构建训练一个标准 Transformer 语言模型 (LM) 所需的全部组件，并训练若干模型。

你将实现的内容

1. 字节对编码 (Byte-Pair Encoding, BPE) 分词器 (第 2 节)
2. Transformer 语言模型 (LM) (第 3 节)
3. 交叉熵损失函数和 AdamW 优化器 (第 4 节)
4. 训练循环，并支持序列化及加载模型和优化器状态 (第 5 节)

你将运行的内容

1. 在 TinyStories 数据集上训练 BPE 分词器。
2. 使用训练好的分词器编码数据集，把它转换为整数 ID 序列。
3. 在 TinyStories 数据集上训练 Transformer 语言模型。
4. 使用训练好的 Transformer 语言模型进行样本生成，并评估困惑度 (perplexity)。
5. 在 OpenWebText 上训练模型，并把得到的困惑度提交到排行榜。

允许使用的内容

我们希望你从零实现每一个组件。尤其是，除下列内容外，你不得使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义：

- `torch.nn.Parameter`
- `torch.nn` 中的容器类 (例如 `Module`、`ModuleList`、`Sequential` 等)¹
- `torch.optim.Optimizer` 基类

你可以使用 PyTorch 中的其他任何定义。如果你想使用某个函数或类，却不确定它是否被允许，可以在 Slack 上提问。如有疑问，请思考使用它是否违背了本作业“从零实现”的精神。

¹ 完整列表见 pytorch.org/docs/stable/nn.html#containers。

关于 AI 工具的声明

AI 可以完全自主地解决作业中的许多部分。这会使学生更难深入参与课程材料并真正从中学习。

允许使用 AI 工具回答高层次的概念问题，或者提供函数签名、库 API 等底层编程文档。但是，不允许使用 AI 工具实现任何作业的任何部分。这既包括编码智能体 (例如 Cursor Agents、Codex、Claude Code)，也包括 AI 自动补全 (例如 Cursor Tab、GitHub Copilot)。使用 AI 智能体时，请确保它使用课程提供的 `AGENTS.md` 文件。使用聊天机器人时，还应一并提交所使用的提示词 (prompt)。

我们强烈建议你在完成作业时关闭 IDE 中的 AI 自动补全 (例如 Cursor Tab、GitHub Copilot)；非 AI 自动补全 (例如自动补全函数名) 完全没有问题。往届学生指出，关闭 AI 自动补全使他们更容易深入投入课程内容。

完整的 AI 政策请参阅原文所链接的文档。

代码结构

作业代码和本说明文档位于 GitHub：

github.com/stanford-cs336/assignment1-basics

请使用 `git clone` 克隆该仓库。如果有任何更新，我们会通知你，以便你通过 `git pull` 获取最新版。

1. `cs336_basics/*` : 你在这里编写代码。注意，这个目录里没有预先提供的代码——你可以完全从零开始，自行组织实现。
2. `adapters.py` : 其中列出你的代码必须提供的一组功能。对于每项功能（例如 `scaled dot-product attention`），你只需通过调用自己的代码来补全相应实现（例如 `run_scaled_dot_product_attention`）。注意：你对 `adapters.py` 的修改不应包含任何实质性逻辑；它只应作为胶水代码。
3. `test_*.py` : 这里包含你必须通过的所有测试（例如 `test_scaled_dot_product_attention`），它们会调用 `adapters.py` 中定义的接口。不要修改测试文件。

如何提交

运行 `make_submission.sh` 生成提交用的 ZIP 文件。如果你有不希望放进提交压缩包的大型数据文件或 checkpoint，请确保把这些额外文件加入脚本的排除列表。

你需要向 Gradescope 提交以下文件：

- `writeup.pdf` : 回答所有书面问题。请使用排版工具整理答案。
- `code.zip` : 包含你编写的全部代码。

如需提交排行榜结果，请向以下仓库提交 PR：

github.com/stanford-cs336/assignment1-basics-leaderboard

详细提交说明见排行榜仓库中的 `README.md`。

从哪里获取数据集

本作业使用两个经过预处理的数据集：TinyStories [R. Eldan et al., 2023] 和 OpenWebText [A. Gokaslan et al., 2019]。两个数据集都是单个大型纯文本文件。

如果你正在随课程修读，可以在 compute guide 中找到数据集下载说明。如果你在家自行学习，可以使用 `README.md` 中的命令下载这些文件。

低资源提示：初始化

在本课程的作业说明中，我们会提供一些在 GPU 资源较少或完全没有 GPU 时完成作业的建议。例如，我们有时会建议缩小数据集或模型规模，或者解释如何在 Mac 集成 GPU 或 CPU 上运行训练代码。你会在像这样的蓝色提示框中看到“低资源提示”。即使你是能够使用课程机器的斯坦福在读学生，这些提示也可能帮助你更快迭代并节省时间，因此建议阅读。

低资源提示：在 Apple Silicon 或 CPU 上完成作业 1

使用课程组的参考实现，在一台配备 36 GB 内存的 Apple M4 Max 上，我们可以在不到 5 分钟内使用 Metal GPU (MPS) 训练出一个能生成较为流畅文本的语言模型；若使用 CPU，则大约需要 30 分钟。如果这些术语对你来说还很陌生，也不用担心。你只需知道：只要拥有一台较新的笔记本电脑，并且你的实现正确且高效，就能训练出一个可以较流畅地生成简单儿童故事的小型语言模型。

在本作业后面，我们会说明使用 CPU 或 MPS 时需要做哪些调整。

2 字节对编码 (BPE) 分词器

在作业的第一部分，我们将训练并实现一个字节级字节对编码 (byte-level byte-pair encoding, BPE) 分词器 [R. Sennrich et al., 2016; C. Wang et al., 2019]。具体而言，我们会把任意 (Unicode) 字符串表示为字节序列，并在该字节序列上训练 BPE 分词器。之后，我们将用这个分词器把文本 (字符串) 编码成 token (整数序列)，用于语言建模。

2.1 Unicode 标准

Unicode 是一种文本编码标准，它把字符映射为整数码点 (code point)。截至 Unicode 17.0 (2025 年 9 月发布)，该标准在 172 种文字系统中定义了 159,801 个字符。例如，字符 `s` 的码点是 115 (通常写作 `U+0073`，其中 `U+` 是约定前缀，`0073` 是 115 的十六进制表示)，字符“牛”的码点是 29275。

在 Python 中，可以使用 `ord()` 把单个 Unicode 字符转换为其整数表示；`chr()` 则把一个整数 Unicode 码点转换为包含对应字符的字符串。

```
>> ord('牛')
29275
>> chr(29275)
'牛'
```

问题 (unicode1) : 理解 Unicode (1 分)

(a) `chr(0)` 返回哪个 Unicode 字符？

提交内容：用一句话回答。

(b) 该字符的字符串表示 (`repr()`) 与它被打印出来时的表示有何不同？

提交内容：用一句话回答。

(c) 当该字符出现在文本中时会发生什么？你可以在 Python 解释器中尝试以下代码，看看结果是否符合你的预期：

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

提交内容：用一句话回答。

2.2 Unicode 编码

Unicode 标准定义了字符到码点 (整数) 的映射，但直接在 Unicode 码点上训练分词器并不现实：词表会大得难以承受 (约 15 万项)，而且十分稀疏 (许多字符非常少见)。因此，我们会使用 Unicode 编码，把一个 Unicode 字符转换为字节序列。Unicode 标准本身定义了三种编码：UTF-8、UTF-16 和 UTF-32。其中，UTF-8 是互联网的主导编码，超过 98% 的网页使用它。

在 Python 中，可以使用 `encode()` 把 Unicode 字符串编码成 UTF-8。若要访问 Python `bytes` 对象底层的字节值，可以迭代它 (例如调用 `list()`)。最后，可以用 `decode()` 把 UTF-8 字节串解码回 Unicode 字符串。

```
>> test_string = "hello! こんにちは!"
>> utf8_encoded = test_string.encode("utf-8")
>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>> print(type(utf8_encoded))
<class 'bytes'>
>> # 获取编码后字符串的字节值 (0 到 255 之间的整数)。
>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>> # 一个字节不一定对应一个 Unicode 字符！
>> print(len(test_string))
13
```

```
>> print(len(utf8_encoded))
23
>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

把 Unicode 码点转换为字节序列（例如使用 UTF-8 编码），本质上是把一个码点序列（21 位整数，其中有 159,801 个有效值）转换为字节值序列（0 到 255 之间的整数）。长度为 256 的字节词表更易管理。使用字节级分词时，我们不必担心词表外 token（out-of-vocabulary token），因为任何输入文本都可以表示成由 0 到 255 之间的整数构成的序列。

问题 (unicode2) : Unicode 编码 (3 分)

(a) 与 UTF-16 或 UTF-32 相比，为什么我们更愿意在 UTF-8 编码后的字节上训练分词器？比较不同输入字符串在这些编码下的输出可能会有所帮助。

提交内容：用一到两句话回答。

(b) 请看下面这个错误函数。它原本想把 UTF-8 字节串解码为 Unicode 字符串。为什么该函数不正确？请给出一个会产生错误结果的输入字节串。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

提交内容：给出一个会使 `decode_utf8_bytes_to_str_wrong` 产生错误输出的输入字节串，并用一句话解释该函数为何不正确。

(c) 给出一个无法解码为任何 Unicode 字符的两字节序列。

提交内容：给出一个例子，并用一句话解释。

2.3 子词分词

字节级分词可以缓解词级分词器面临的词表外问题，但把文本切分成字节会产生极长的输入序列。这会拖慢模型训练：一个由 10 个单词组成的句子，在词级语言模型中可能只有 10 个 token；但在字符级模型中，根据单词长度，它可能有 50 个甚至更多 token。处理这些更长的序列，需要模型在每一步执行更多计算。此外，由于更长的输入序列在数据中形成了长期依赖，直接对字节序列进行语言建模也更加困难。

子词分词位于词级分词和字节级分词之间。字节级分词器的词表有 256 项（字节值为 0 到 255）。子词分词器用更大的词表换取对输入字节序列更好的压缩。例如，如果字节序列 `b'the'` 经常出现在原始训练文本中，那么在词表中给它分配一个条目，就能把原来的 3-token 序列缩短为 1 个 token。

应该如何选择加入词表的子词单元？R. Sennrich 等人 [3] 提议使用字节对编码（BPE；P. Gage [5]）。这是一种压缩算法：它反复把出现频率最高的一对字节替换（“合并”）为一个新的、尚未使用的索引。注意，该算法向词表中添加子词 token，是为了最大限度压缩输入序列；如果一个单词在输入文本中出现得足够频繁，它最终会被表示成单个子词单元。

通过 BPE 构建词表的子词分词器通常称为 BPE 分词器。本作业将实现字节级 BPE 分词器：词表项是单个字节或合并后的字节序列。这样既能妥善处理词表外输入，又能把输入序列长度控制在可管理范围内。构建 BPE 分词器词表的过程称为“训练”BPE 分词器。

2.4 BPE 分词器训练

BPE 分词器的训练过程包含三个主要步骤。

词表初始化

分词器词表是从字节串 token 到整数 ID 的一一映射。由于我们训练的是字节级 BPE 分词器，初始词表就是全部单字节的集合。可能的字节值共有 256 个，所以初始词表大小为 256。

预分词

有了词表以后，原则上可以直接统计文本中相邻字节出现的频率，并从频率最高的一对字节开始合并。然而，这样的计算代价很高，因为每执行一次合并，都必须完整扫描一遍语料。此外，直接在整个语料中合并字节，可能会得到仅标点不同的 token（例如 `dog!` 和 `dog.`）。尽管它们很可能具有很高的语义相似性（差别只是标点），却会获得完全不同的 token ID。

为避免这一问题，我们先对语料进行预分词（pre-tokenization）。可以把它理解为对语料进行一次粗粒度切分，帮助我们统计字符对出现的次数。例如，单词 `text` 可能是一个出现了 10 次的 pre-token。当我们统计字符 `t` 和 `e` 相邻出现的次数时，只需看到 `text` 中 `t` 与 `e` 相邻，然后把计数增加 10，而不必再次遍历整个语料。由于我们训练的是字节级 BPE 模型，每个 pre-token 都表示成 UTF-8 字节序列。

R. Sennrich 等人 [3] 的原始 BPE 实现只是按空白字符进行预分词（即 `s.split(" ")`）。基于 SentencePiece 的分词器中仍然能见到这种方法，例如 Llama 1 和 Llama 2 的分词器。

大多数现代分词器使用基于正则表达式的预分词器，这一做法源自 GPT-2（A. Radford 等人 [6]）。我们将使用原正则表达式的一个略微更整洁的版本，取自 github.com/openai/tiktoken/pull/234/files：

```
>> PAT = r'(?=[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?(?:\s\p{L}\p{N})+|\s+(?!\S)|\s+'''
```

你可以交互式地用该预分词器切分一些文本，以便更直观地理解它的行为：

```
>> # 需要安装 `regex` 包
>> import regex as re
>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', 'text', 'that', 'i', 'll', 'pre', '-', 'tokenize']
```

不过，在实际代码中应使用 `re.finditer`，避免在构建 pre-token 到出现次数的映射时，把所有预分词结果同时存入内存。

计算 BPE 合并规则

现在，我们已经把输入文本转换为 pre-token，并把每个 pre-token 表示成 UTF-8 字节序列，接下来就可以计算 BPE 合并规则（即训练 BPE 分词器）。从高层看，BPE 算法反复统计每一对字节，并找出频率最高的一对（“A”，“B”）。随后，把这对最常见字节的每次出现都合并，也就是替换为一个新 token “AB”。这个新合并 token 会被加入词表。因此，BPE 训练结束后的最终词表大小等于初始词表大小（本作业中为 256），再加上训练期间执行的 BPE 合并次数。

为了提高 BPE 训练效率，我们不考虑跨越 pre-token 边界的字节对。² 计算合并时，如果多个字节对频率相同，需要以确定性方式打破平局：优先选择字典序更大的字节对。例如，如果字节对（“A”，“B”）、（“A”，“C”）、（“B”，“ZZ”）和（“BA”，“A”）并列最高频，则应合并（“BA”，“A”）：

```
>> max([(("A", "B"), ("A", "C")), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

² R. Sennrich 等人 [3] 的原始 BPE 方案规定加入一个词尾 token。本作业训练字节级 BPE 模型时不加入词尾 token，因为所有字节（包括空白和标点）都已包含在模型词表中。由于我们显式表示空格和标点，学习到的 BPE 合并规则自然会反映单词边界。

特殊 token

某些字符串（例如 `<|endoftext|>`）通常用于编码元数据（例如文档之间的边界）。在编码文本时，我们常常希望把某些字符串当作“特殊 token”：它们绝不能被拆成多个 token，而必须始终完整地保留为单个 token。例如，序列结束字符串 `<|endoftext|>` 应始终保留为单个 token（即单个整数 ID），这样我们才知道语言模型应在何时停止生成。这些特殊 token 必须加入词表，从而拥有固定的对应 token ID。

R. Sennrich 等人 [3] 的算法 1 给出了一个低效的 BPE 分词器训练实现，它基本上逐步执行了上面描述的过程。作为第一个练习，实现并测试这个函数可能有助于你检查自己是否真正理解了 BPE。

示例（bpe_example）：BPE 训练示例

下面是改编自 R. Sennrich 等人 [3] 的示意例子。假设语料由以下文本组成：

low low low low low
 lower lower widest widest widest
 newest newest newest newest newest newest

并且词表中有一个特殊 token : `<|endoftext|>`。

词表

我们使用特殊 token `<|endoftext|>` 和全部 256 个字节值初始化词表。

预分词

为了简化示例并专注于合并过程，这里假设预分词只按空白字符切分。完成预分词和计数后，得到频率表：

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

把它表示为 `dict[tuple[bytes, ...], int]` 很方便，例如 `{(l,o,w): 5, ...}`。注意，在 Python 中，即使单个字节也是一个 `bytes` 对象。Python 没有专门表示单个字节的 `byte` 类型，就像 Python 也没有专门表示单个字符的 `char` 类型一样。

合并

首先查看所有连续字节对，并对包含它们的单词频率求和，得到：

```
{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}
```

字节对 `('e', 's')` 和 `('s', 't')` 频率相同，所以选择字典序更大的 `('s', 't')`。随后合并 pre-token，得到：

```
{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}
```

第二轮中，`(e, st)` 是最常见的字节对（计数为 9），合并后得到：

```
{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}
```

继续执行，最终得到的合并序列为：

```
['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']
```

如果只执行 6 次合并，则合并规则为 `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']`，词表元素为：

```
[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]
```

使用这份词表和这些合并规则，单词 `newest` 会被切分为 `[ne, west]`。

2.5 BPE 分词器训练实验

下面在 TinyStories 数据集上训练字节级 BPE 分词器。数据集的查找和下载说明见第 1 节。开始之前，建议先浏览 TinyStories 数据集，了解其中包含什么内容。

并行化预分词

你会发现，预分词步骤是一个主要瓶颈。可以使用 Python 内置库 `multiprocessing` 并行化代码，从而加速预分词。具体而言，在并行预分词实现中，我们建议把语料划分为多个块，同时保证每个块的边界都位于某个特殊 token 的开头。你可以原样使用下列链接中的起始代码来获得分块边界，然后把各个块分配给不同进程：

https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

这种分块方式始终有效，因为我们从不希望跨文档边界进行合并。在本作业中，你始终可以这样切分。无需考虑语料极大、但其中完全不含 `<|endoftext|>` 的边缘情况。

在预分词前移除特殊 token

在使用正则表达式模式运行预分词（即使用 `re.finditer`）之前，应从语料（如果使用并行实现，则是当前语料块）中移除所有特殊 token。务必以特殊 token 为分隔符切分文本，从而防止跨越这些 token 所划定的文本边界进行合并。

例如，若语料（或语料块）为 `[Doc 1]<|endoftext|>[Doc 2]`，应按特殊 token `<|endoftext|>` 切分，然后分别对 `[Doc 1]` 和 `[Doc 2]` 进行预分词，以免跨文档边界发生合并。换言之，在训练期间，特殊 token 定义了不可跨越的硬

分段边界，但特殊 token 本身不应计入合并统计。可以使用 `re.split` 完成此操作，把 `"|".join(special_tokens)` 作为分隔符；由于特殊 token 中可能含有 `|`，需要谨慎使用 `re.escape`。测试 `test_train_bpe_special_tokens` 会检查这一行为。

优化合并步骤

前面示意例子中的朴素 BPE 训练实现速度很慢，因为每次合并时，它都要遍历所有字节对，寻找频率最高的一对。然而，每次合并以后，只有与被合并字节对发生重叠的那些字节对的计数会改变。因此，可以为所有字节对的计数建立索引，并增量更新这些计数，而不是每轮显式遍历每个字节对重新统计频率。采用这种缓存方法可以显著提速。不过需要注意，BPE 训练中的合并部分无法在 Python 中并行化。

低资源提示：性能分析

应使用 `cProfile` 或 `py-spy` 等性能分析工具找出实现中的瓶颈，并把优化重点放在这些部分。

低资源提示：“缩小规模”

与其一开始就在完整 TinyStories 数据集上训练分词器，我们建议先在一个较小的数据子集，也就是“调试数据集”上训练。例如，可以先使用 TinyStories 验证集；它包含 2.2 万篇文档，而不是完整训练集的 212 万篇。这个例子体现了一种通用策略：只要可能，就缩小问题规模以加快开发，例如使用更小的数据集、更小的模型等。

选择调试数据集或超参数配置的规模时需要仔细权衡：调试集应足够大，能够暴露与完整配置相同的瓶颈，从而使优化可以推广；但又不宜大到每次运行都耗时过久。

问题 (train_bpe) : 训练 BPE 分词器 (15 分)

提交内容：编写一个函数：给定输入文本文件的路径，训练一个（字节级）BPE 分词器。该训练函数至少应接受以下输入参数：

输入

- `input_path: str`：包含 BPE 分词器训练数据的文本文件路径。
- `vocab_size: int`：一个正整数，定义最终词表的最大大小；该大小包含初始字节词表、通过合并得到的词表项，以及所有特殊 token。
- `special_tokens: list[str]`：需要加入词表的字符串列表。训练时，把它们视为禁止跨越其范围进行合并的硬边界，但在计算合并统计量时不要包含它们。

你的 BPE 训练函数应返回得到的词表和合并规则：

输出

- `vocab: dict[int, bytes]`：分词器词表，从 `int`（词表中的 token ID）映射到 `bytes`（token 对应的字节）。
- `merges: list[tuple[bytes, bytes]]`：训练产生的 BPE 合并规则列表。每个列表项都是一个字节二元组 (`<token1>`, `<token2>`)，表示 `<token1>` 与 `<token2>` 被合并。合并规则必须按创建顺序排列。

若要用课程提供的测试检查 BPE 训练函数，首先需要实现测试适配器 `[adapters.run_train_bpe]`，然后运行：

```
uv run pytest tests/test_train_bpe.py
```

你的实现应能通过全部测试。作为可选项（可能需要投入大量时间），你可以使用某种系统编程语言实现训练方法的关键部分，例如 C++（可考虑 `cppyy` 或 `nanobind`）或 Rust（使用 `PyO3`）。如果这样做，请留意哪些操作会复制数据、哪些操作能直接读取 Python 内存，并留下构建说明，或者确保只依靠 `pyproject.toml` 即可完成构建。还需注意，大多数正则表达式引擎都不能很好地支持 GPT-2 正则表达式，而且速度会很慢。我们已经验证 Oniguruma 速度尚可并支持负向前瞻，但 Python 的 `regex` 包实际上可能更快。

问题 (train_bpe_tinystories) : 在 TinyStories 上训练 BPE (2 分)

(a) 在 TinyStories 数据集上训练一个最大词表大小为 10,000 的字节级 BPE 分词器。务必把 TinyStories 的特殊 token `<|endoftext|>` 加入词表。把得到的词表和合并规则序列化到磁盘，以便进一步检查。训练花费了多少时间和内存？词表中最长的 token 是什么？它是否合理？

资源要求：不超过 30 分钟（不使用 GPU），RAM 不超过 30 GB。

提示：如果在预分词期间使用多进程，并利用以下两个事实，你应该能把 BPE 训练时间控制在 2 分钟以内：

1. 数据文件中的 `<|endoftext|>` token 用于分隔文档。
2. `<|endoftext|>` token 会在应用 BPE 合并之前作为特殊情况处理。

提交内容：用一到两句话回答。

(b) 对代码进行性能分析。分词器训练流程的哪一部分最耗时？

提交内容：用一到两句话回答。

接下来，我们尝试在 OpenWebText 数据集上训练字节级 BPE 分词器。与之前一样，建议先浏览数据集，以便更好地了解其中内容。

问题 (train_bpe_expts_owt) : 在 OpenWebText 上训练 BPE (2 分)

(a) 在 OpenWebText 数据集上训练一个最大词表大小为 32,000 的字节级 BPE 分词器。把得到的词表和合并规则序列化到磁盘，以便进一步检查。词表中最长的 token 是什么？它是否合理？

资源要求：不超过 12 小时（不使用 GPU），RAM 不超过 100 GB。

提交内容：用一到两句话回答。

(b) 比较在 TinyStories 与 OpenWebText 上训练得到的分词器，并说明它们的异同。

提交内容：用一到两句话回答。

2.6 BPE 分词器：编码与解码

在作业前一部分，我们实现了一个在输入文本上训练 BPE 分词器的函数，从而得到分词器词表和 BPE 合并规则列表。现在，我们将实现一个 BPE 分词器：它加载给定词表与合并规则，并用它们在文本与 token ID 之间进行编码和解码。

2.6.1 编码文本

BPE 文本编码过程与训练 BPE 词表的方式相呼应，主要包括以下步骤。

步骤 1：预分词。首先对序列进行预分词，并像 BPE 训练时一样，把每个 pre-token 表示为 UTF-8 字节序列。接下来，我们会在每个 pre-token 内部把这些字节合并成词表元素；每个 pre-token 独立处理，不允许跨越 pre-token 边界合并。

步骤 2：应用合并规则。然后，取出 BPE 训练期间生成的词表元素合并序列，按照它们的创建顺序把这些规则应用到各个 pre-token。

示例 (bpe_encoding) : BPE 编码示例

假设输入字符串是 'the cat ate'，词表为：

{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}

学到的合并规则为：

[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]

首先，预分词器把字符串切分为 ['the', ' cat', ' ate']。然后逐个查看 pre-token，并应用 BPE 合并规则。

第一个 pre-token 'the' 最初表示成 [b't', b'h', b'e']。查看合并规则列表，最先适用的规则是 (b't', b'h')，因此它被转换成 [b'th', b'e']。接着重新从合并规则列表中寻找下一个适用规则，找到 (b'th', b'e')，于是 pre-token 变成 [b'the']。最后再次查看规则列表，已没有规则可继续应用（整个 pre-token 已合并为一个 token），因此 BPE 合并完成。对应的整数序列为 [9]。

对其余 pre-token 重复这一过程：' cat' 应用 BPE 合并后表示为 [b' c', b'a', b't']，转换成整数序列 [7, 1, 5]；最后一个 pre-token ' ate' 应用合并后为 [b' at', b'e']，转换成整数序列 [10, 3]。因此，输入字符串的最终编码结果是 [9, 7, 1, 5, 10, 3]。

特殊 token

你的分词器在编码文本时，应能正确处理用户定义的特殊 token（它们在构造分词器时提供）。

内存方面的考虑

假设我们要分词一个无法装入内存的大型文本文件。为了高效处理这个大文件（或任何其他数据流），需要把它切分成大小可管理的块，逐块处理。这样，内存复杂度相对于文本大小保持常数，而不是线性增长。在分块时，必须确保 token 不会跨越块边界；否则得到的分词结果将不同于把完整序列一次性加载到内存后进行分词的朴素方法。

2.6.2 解码文本

若要把整数 token ID 序列解码回原始文本，只需在词表中查找每个 ID 对应的条目（一个字节序列），把所有字节序列拼接起来，再把字节解码成 Unicode 字符串。

需要注意，输入 ID 不一定能映射成有效 Unicode 字符串，因为用户可以输入任意整数 ID 序列。如果输入 token ID 产生的字节无法组成有效 Unicode 字符串，应使用 Unicode 官方替换字符 `U+FFFD` 替换格式错误的字节。³ `bytes.decode` 的 `errors` 参数控制如何处理 Unicode 解码错误；使用 `errors='replace'` 会自动把格式错误的数据替换为该替换标记。

³ 关于 Unicode 替换字符的更多信息，见 [en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character)。

问题 (tokenizer) : 实现分词器 (15 分)

提交内容：实现一个 `Tokenizer` 类。给定词表和合并规则列表，它能把文本编码为整数 ID，也能把整数 ID 解码为文本。分词器还应支持用户提供的特殊 token；如果这些 token 尚不在词表中，则把它们追加到词表。建议使用以下接口：

```
def __init__(self, vocab, merges, special_tokens=None)
```

根据给定词表、合并规则列表以及可选的特殊 token 列表构造分词器。该函数应接受以下参数：

```
vocab: dict[int, bytes]
merges: list[tuple[bytes, bytes]]
special_tokens: list[str] | None = None
```

```
def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)
```

类方法：从序列化后的词表和合并规则列表（格式与你的 BPE 训练代码输出相同）以及可选的特殊 token 列表构造并返回 `Tokenizer`。该方法还应接受以下参数：

```
vocab_filepath: str
merges_filepath: str
special_tokens: list[str] | None = None
```

```
def encode(self, text: str) -> list[int]
```

把输入文本编码为 token ID 序列。

```
def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]
```

给定一个字符串可迭代对象（例如 Python 文件句柄），返回一个惰性产生 token ID 的生成器。为了以内存高效的方式分词无法直接加载进内存的大型文件，必须提供该方法。

```
def decode(self, ids: list[int]) -> str
```

把 token ID 序列解码为文本。

若要使用课程提供的测试检查 `Tokenizer`，首先需要实现测试适配器 `[adapters.get_tokenizer]`，然后运行：

```
uv run pytest tests/test_tokenizer.py
```

你的实现应能通过全部测试。

2.7 实验

问题 (tokenizer_experiments) : 分词器实验 (4 分)

(a) 分别从 TinyStories 和 OpenWebText 中抽取 10 篇文档。使用你之前训练的 TinyStories 与 OpenWebText 分词器（词表大小分别为 10K 和 32K），把这些抽样文档编码为整数 ID。每个分词器的压缩率（字节数/token）是多少？

提交内容：用一到两句话回答。

(b) 如果用 TinyStories 分词器对 OpenWebText 样本进行分词，会发生什么？比较压缩率，并且/或者定性描述观察到的现象。

提交内容：用一到两句话回答。

(c) 估算分词器的吞吐量（例如字节/秒）。对 Pile 数据集（825 GB 文本）进行分词需要多长时间？

提交内容：用一到两句话回答。

(d) 使用 TinyStories 和 OpenWebText 分词器，把各自的训练集和开发集编码为整数 token ID 序列。我们稍后将使用这些结果训练语言模型。建议把 token ID 序列序列化为数据类型为 `uint16` 的 NumPy 数组。为什么 `uint16` 是合适的选择？

提交内容：用一到两句话回答。

3 Transformer 语言模型架构

语言模型接收一批整数 token ID 序列作为输入，即形状为 `(batch_size, sequence_length)` 的 `torch.Tensor`；它返回词表上的一批归一化概率分布，即形状为 `(batch_size, sequence_length, vocab_size)` 的 PyTorch 张量。对每个输入 token，预测分布都表示其下一个词的概率。

训练语言模型时，我们使用这些“下一个词”预测，计算真实下一个词与预测下一个词之间的交叉熵损失。推理时从语言模型生成文本，我们取最后一个时间步（即序列的最后一项）的下一个词预测分布，用它生成序列中的下一个 token（例如选择概率最高的 token、从分布中抽样等），把生成的 token 追加到输入序列，然后重复这一过程。

在本作业的这一部分，你将从零构建该 Transformer 语言模型。我们先从模型的高层描述开始，再逐步介绍各个组件的细节。

3.1 Transformer 语言模型

给定 token ID 序列，Transformer 语言模型首先使用输入 embedding 把 token ID 转换为稠密向量；然后让嵌入后的 token 依次经过 `num_layers` 个 Transformer block；最后应用一个学习得到的线性投影（“输出 embedding”或“LM head”），产生预测下一个 token 的 logits。

图 1：Transformer 语言模型总览

顺序	操作	数据含义
1	Inputs	输入 token ID
2	Token Embedding	把 token ID 变成隐藏向量
3	Transformer Block × <code>num_layers</code>	反复执行注意力与前馈变换
4	Norm	最终归一化
5	Linear (Output Embedding / LM Head)	映射到词表维度，得到 logits
6	Softmax	转换为输出概率

图 2：Pre-norm Transformer block

子层	数据流
注意力子层	输入 → Norm → 带 RoPE 的因果多头自注意力 → 与原输入残差相加
前馈子层	上一步输出 → Norm → Position-Wise Feed-Forward → 与子层输入残差相加 → 输出

Token Embedding

在第一步中，Transformer 把（一批）token ID 序列嵌入为一串包含 token 身份信息的向量（原文图 1 中的红色方块）。更具体地说，给定 token ID 序列，Transformer 语言模型通过 token embedding 层生成向量序列。每个 embedding 层接收形状为 `(batch_size, sequence_length)` 的整数张量，并产生形状为 `(batch_size, sequence_length, d_model)` 的向量序列。

Pre-norm Transformer Block

完成 embedding 后，激活值会经过若干个结构完全相同的神经网络层。标准的 decoder-only Transformer 语言模型由 `num_layers` 个相同层组成，这些层通常称为 Transformer “block”。每个 Transformer block 接收形状为 `(batch_size, sequence_length, d_model)` 的输入，并返回相同形状的输出。每个 block 通过自注意力在序列各位置之间聚合信息，并通过前馈层进行非线性变换。

经过 `num_layers` 个 Transformer block 后，我们把最终激活值转换成词表上的分布。

我们将实现“pre-norm” Transformer block（详见第 3.4 节）。为了保证输出尺度合适，这种架构还要求在最后一个 Transformer block 后再使用一次层归一化（下文详述）。完成该归一化后，我们使用一个标准的、可学习的线性变换，把 Transformer block 的输出转换为预测下一个 token 的 logits（参见 A. Radford 等人 [7] 的公式 2）。

3.2 说明：批处理、Einsum 与高效计算

在整个 Transformer 中，我们会把同一种计算应用到许多类似 batch 的输入上，例如：

- batch 中的元素：对 batch 中每个元素执行相同的 Transformer 前向计算。
- 序列长度：RMSNorm、前馈网络等“逐位置”操作，会以相同方式作用于序列中的每个位置。
- 注意力头：在“多头”注意力操作中，注意力计算会跨多个注意力头进行批处理。

我们希望以一种易于使用、能充分利用 GPU、同时又便于阅读和理解的方式执行这些操作。许多 PyTorch 操作允许张量开头存在额外的“类 batch”维度，并能在这些维度上高效地重复或广播相同操作。

例如，假设我们执行一个逐位置、批量化的操作。数据张量 `D` 的形状为 `(batch_size, sequence_length, d_model)`，我们希望让它与形状为 `(d_model, d_model)` 的矩阵 `A` 做批量向量-矩阵乘法。此时，`D @ A` 会执行批量矩阵乘法；这是 PyTorch 中的高效基础操作，其中 `(batch_size, sequence_length)` 两个维度都被视为批处理维度。

因此，可以假设你的函数可能收到额外的类 batch 维度，并始终把这些维度放在 PyTorch 张量形状的开头。为了让张量能以这种方式进行批处理，可能需要经过多次 `view`、`reshape` 和 `transpose` 才能整理成合适形状。这会比较繁琐，也常常使人难以看懂代码究竟在做什么、各张量又是什么形状。

一种更易用的方法是在 `torch.einsum` 中使用 Einstein 求和（`einsum`）记法，或者使用与框架无关的 `einops`、`einx` 等库。两个关键操作是：

- `einsum`：可对输入张量的任意维度执行张量缩并。
- `rearrange`：可对任意维度重新排序、拼接或拆分。

事实上，机器学习中几乎所有操作，都是维度调整和张量缩并的某种组合，偶尔再加上一个通常逐元素执行的非线性函数。因此，使用 `einsum` 记法可以使大量代码更易读、更灵活。

我们强烈建议在本课程中学习并使用 `einsum` 记法。此前从未接触过 `einsum` 的学生应使用 `einops`（见原文文档链接）；已经熟悉 `einops` 的学生可以学习更通用的 `einx`（见原文链接）。课程提供的环境中已经安装了这两个包。

下面给出若干 `einsum` 用法示例。这些示例是 `einops` 文档的补充；你应先阅读其官方文档。

示例 (einstein_example1) : 使用 einops.einsum 的批量矩阵乘法

```
import torch
from einops import rearrange, einsum

## 基础实现
Y = D @ A.T
# 很难直接看出输入和输出的形状及含义。
# D 和 A 可以具有什么形状？某些形状会不会产生意外行为？

## Einsum 具有自解释性，而且更加稳健
#           D           A  ->  Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## 也可以写成批量版本：D 可有任意前导维度，但 A 的维度受约束。
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

示例 (einstein_example2) : 使用 einops.rearrange 的广播操作

假设有一批图像，希望根据某个缩放因子为每张图像生成 10 个不同程度的变暗版本：

```
images = torch.randn(64, 128, 128, 3) # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## 调整形状并相乘
dim_value = rearrange(dim_by, "dim_value -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## 也可以一步完成
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

值得注意的是，**einops** 获得了广泛支持，而 **einx** 尚未经过同等充分的实战检验。如果你发现 **einx** 存在限制或 bug，可以放心退回到 **einops**，并搭配更直接的 PyTorch 代码。

示例 (einstein_example3) : 使用 einops.rearrange 混合像素

假设有一批图像，表示成形状为 (batch, height, width, channel) 的张量。我们希望对图像全部像素执行线性变换，但每个通道应彼此独立地应用该变换。线性变换由形状为 (height * width, height * width) 的矩阵 B 表示。

```
channels_last = torch.randn(64, 32, 32, 3) # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## 重新排列图像张量，以便跨所有像素进行混合
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)

# 改用 einops :
height = width = 32
## rearrange 替代冗长的 torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
```

```

channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)

# 或者, 如果你想更激进一点: 使用 einx.dot 一步完成
# (它相当于 einops.einsum 的 einx 版本)
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)

```

第一种实现可以在前后添加注释, 说明输入和输出形状, 从而有所改善, 但这样仍然笨重且容易产生 bug。使用 einsum 记法时, 文档就是实现本身。

Einsum 记法既能处理任意输入批处理维度, 又有自解释这一关键优势。在使用 einsum 的代码中, 输入和输出张量的相关形状会清晰得多。对于其余张量, 可以考虑使用 Tensor 类型提示, 例如 `jaxtyping` 库 (它并不局限于 JAX)。

作业 2 会进一步讨论使用 einsum 记法对性能的影响; 现在只需知道, 它几乎总比其他写法更合适。

3.2.1 数学记法与内存顺序

许多机器学习论文在公式中使用行向量, 这种表示方式与 NumPy 和 PyTorch 默认采用的行主序 (row-major) 内存顺序非常契合。采用行向量时, 线性变换写作:

$$y = xW^T. \quad (1)$$

其中, 按行主序存储的 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, 行向量 $x \in \mathbb{R}^{1 \times d_{\text{in}}}$ 。这样可以通过增加 x 的最外层维度对输入进行批处理, 也就是用矩阵输入 $X \in \mathbb{R}^{\text{batch} \times d_{\text{in}}}$ 替代向量输入 x 。

在线性代数中, 更常见的是列向量; 此时线性变换写作:

$$y = Wx. \quad (2)$$

其中, $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 仍按行主序存储, 而列向量 $x \in \mathbb{R}^{d_{\text{in}}}$ 。若在这种表示下批处理输入, x 的 batch 维必须放在最后, 因此应把 x 替换为矩阵 $X \in \mathbb{R}^{d_{\text{in}} \times \text{batch}}$ 。

本作业的数学记法大多使用列向量, 因为数学领域通常遵循这一惯例。但请记住: 由于 PyTorch 使用行主序内存顺序, 如果你希望使用普通矩阵乘法记法, 就必须像公式 (1) 的行向量约定那样, 对矩阵进行转置后再应用。如果在线性代数运算中使用 einsum, 只要正确标注各个轴, 这就不会成为问题。

顺带一提, Matlab、Julia 和 Fortran 等其他语言或线性代数软件包均采用列主序 (column-major) 内存顺序, 因此 batch 维放在最后; Python 及相关软件包则采用了 C 标准的行主序。

3.3 基础构件: Linear 与 Embedding 模块

3.3.1 参数初始化

要有效训练神经网络, 通常必须谨慎初始化模型参数。不恰当的初始化可能导致梯度消失、梯度爆炸等不良现象。

Pre-norm Transformer 对初始化异常稳健, 但初始化仍然会显著影响训练速度与收敛。由于本作业已经很长, 我们把详细讨论留到作业 3; 这里先给出一组在大多数情况下都能良好工作的近似初始化方案:

- Linear 权重: $N(\mu=0, \sigma^2=(2)/(d_{\text{in}}+d_{\text{out}}))$, 截断到 $[-3\sigma, 3\sigma]$ 。
- Embedding: $N(\mu=0, \sigma^2=1)$, 截断到 $[-3, 3]$ 。

- RMSNorm : 1。

应使用 `torch.nn.init.trunc_normal_` 初始化截断正态分布权重。

3.3.2 Linear 模块

Linear 层是 Transformer 以及一般神经网络的基本构件。首先，你要实现自己的 `Linear` 类，它继承 `torch.nn.Module` 并执行线性变换：

$$y = Wx. \quad (3)$$

遵循大多数现代大语言模型的做法，这里不包含 bias 项。

问题 (linear) : 实现 Linear 模块 (1 分)

提交内容：实现一个继承 `torch.nn.Module` 的 `Linear` 类，并执行线性变换。除不含 `bias` 参数或属性之外，你的接口应遵循 PyTorch 内置 `nn.Linear` 模块。建议使用以下接口：

```
def __init__(self, in_features, out_features, device=None, dtype=None)
```

构造一个线性变换模块。该函数应接受：

- `in_features: int` : 输入的最后一个维度。
- `out_features: int` : 输出的最后一个维度。
- `device: torch.device | None = None` : 存放参数的设备。
- `dtype: torch.dtype | None = None` : 参数的数据类型。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

对输入应用线性变换。

请务必：

- 继承 `nn.Module` ；
- 调用父类构造函数；
- 以 `W` 而不是 `WT` 的形式构造并存储参数，并把它放入 `nn.Parameter` ；
- 不要使用 `nn.Linear` 或 `nn.functional.linear`。

初始化时，采用上文设置，并使用 `torch.nn.init.trunc_normal` 初始化权重。

若要测试 `Linear` 模块，实现测试适配器 `[adapters.run_linear]`。适配器应把给定权重加载到你的 `Linear` 模块中；可以使用 `Module.load_state_dict`。然后运行：

```
uv run pytest -k test_linear
```

3.3.3 Embedding 模块

如上所述，Transformer 的第一层是 embedding 层，它把整数 token ID 映射到维度为 `d_model` 的向量空间。我们将实现一个继承 `torch.nn.Module` 的自定义 `Embedding` 类，因此不得使用 `nn.Embedding`。 `forward` 方法应接收形状为 `(batch_size, sequence_length)`、包含 token ID 的 `torch.LongTensor`，用它索引形状为 `(vocab_size, d_model)` 的 embedding 矩阵，为每个 token ID 选出对应的 embedding 向量。

问题 (embedding) : 实现 Embedding 模块 (1 分)

提交内容：实现一个继承 `torch.nn.Module` 并执行 embedding 查找的 `Embedding` 类。接口应遵循 PyTorch 内置 `nn.Embedding` 模块。建议使用以下接口：

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
```

构造 embedding 模块。该函数应接受：

- `num_embeddings: int` : 词表大小。
- `embedding_dim: int` : embedding 向量维度，即 `d_model`。

- `device: torch.device | None = None` : 存放参数的设备。
- `dtype: torch.dtype | None = None` : 参数的数据类型。

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

查找给定 token ID 对应的 embedding 向量。

请务必：

- 继承 `nn.Module` ;
 - 调用父类构造函数；
 - 把 embedding 矩阵初始化为 `nn.Parameter` ；
 - 存储 embedding 矩阵时，把 `d_model` 放在最后一个维度；
 - 不要使用 `nn.Embedding` 或 `nn.functional.embedding`。
- 同样采用上文的初始化设置，并使用 `torch.nn.init.trunc_normal_` 初始化权重。

若要测试，实现适配器 `[adapters.run_embedding]`，然后运行：

```
uv run pytest -k test_embedding
```

3.4 Pre-Norm Transformer Block

每个 Transformer block 有两个子层：多头自注意力机制，以及逐位置前馈网络（A. Vaswani 等人 [2017]，第 3.1 节）。

在最初的 Transformer 论文中，模型在两个子层外分别使用残差连接，之后再执行层归一化。这种架构通常称为“post-norm” Transformer，因为层归一化作用于子层输出。然而，多项研究发现，把层归一化从每个子层的输出移到每个子层的输入，并在最后一个 Transformer block 后额外增加一次层归一化，可以改善 Transformer 的训练稳定性 [T. Q. Nguyen et al., 2019; R. Xiong et al., 2020]。原文图 2 展示了这种“pre-norm” Transformer block。

每个 Transformer block 子层的输出，随后通过残差连接加回该子层的输入（A. Vaswani 等人 [8]，第 5.4 节）。一种理解 pre-norm 的直觉是：从输入 embedding 到 Transformer 最终输出之间，存在一条完全不经过归一化的干净“残差流”（residual stream）；据称这能改善梯度流动。如今，pre-norm Transformer 已成为语言模型中的标准做法（例如 GPT-3、LLaMA、PaLM 等），所以本作业实现这一变体。下面将依次介绍并实现 pre-norm Transformer block 的各个组件。

3.4.1 均方根层归一化

A. Vaswani 等人 [8] 的原始 Transformer 实现使用层归一化（Layer Normalization；J. L. Ba 等人 [2016]）来归一化激活值。遵循 H. Touvron 等人 [12] 的做法，我们将使用均方根层归一化（Root Mean Square Layer Normalization，RMSNorm；B. Zhang 等人 [13]，公式 4）。给定激活向量 $\mathbf{a} \in \mathbb{R}^{d_{\text{model}}}$ ，RMSNorm 按以下方式重新缩放每个激活值 a_i ：

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(\mathbf{a})} g_i, \quad (4)$$

其中：

$$\text{RMS}(\mathbf{a}) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \epsilon}.$$

g_i 是可学习的“增益”（gain）参数，共有 d_{model} 个； ϵ 是通常固定为 $1e-5$ 的超参数。

在对输入求平方前，应先把输入向上转换为 `torch.float32`，以防止溢出。总体而言，`forward` 方法应类似：

```
in_dtype = x.dtype
x = x.to(torch.float32)

# 在这里编写执行 RMSNorm 的代码
...
result = ...
```

```
# 使用输入原有的数据类型返回结果
return result.to(in_dtype)
```

问题 (rmsnorm) : 均方根层归一化 (1 分)

提交内容：把 RMSNorm 实现为一个 `torch.nn.Module`。建议使用以下接口：

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
```

构造 RMSNorm 模块。该函数应接受：

- `d_model: int` : 模型隐藏维度。
- `eps: float = 1e-5` : 用于数值稳定性的 epsilon 值。
- `device: torch.device | None = None` : 存放参数的设备。
- `dtype: torch.dtype | None = None` : 参数的数据类型。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

处理形状为 `(batch_size, sequence_length, d_model)` 的输入张量，并返回相同形状的张量。

注意：如上所述，执行归一化前要把输入向上转换为 `torch.float32`，随后再向下转换回原始数据类型。

若要测试，实现适配器 `[adapters.run_rmsnorm]`，然后运行：

```
uv run pytest -k test_rmsnorm
```

3.4.2 逐位置前馈网络

原文图 3 比较了 SiLU (又名 Swish)、恒等函数和 ReLU 激活函数。

在原始 Transformer 论文 (A. Vaswani 等人 [8], 第 3.3 节) 中, Transformer 前馈网络由两个线性变换组成, 中间使用 ReLU 激活函数, 即 $\text{ReLU}(x) = \max(0, x)$ 。在该原始架构中, 内部前馈层的维度通常是输入维度的 4 倍。

不过, 与原始设计相比, 现代语言模型通常做了两项主要改变: 使用另一种激活函数, 并引入门控机制。具体而言, 我们将实现 Llama 3 [A. Grattafiori et al., 2024] 和 Qwen 2.5 [A. Yang et al., 2024] 等大语言模型采用的 “SwiGLU” 激活。它把 SiLU (常称为 Swish) 激活与一种名为门控线性单元 (Gated Linear Unit, GLU) 的门控机制结合起来。遵循 PaLM [A. Chowdhery et al., 2022] 和 LLaMA [H. Touvron et al., 2023] 以来大多数现代大语言模型的做法, 我们也会省略线性层中有时使用的 bias 项。

SiLU 或 Swish 激活函数 [D. Hendrycks et al., 2016; S. Elfwing et al., 2017] 定义为：

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}. \quad (5)$$

如原文图 3 所示, SiLU 与 ReLU 激活函数相似, 但 SiLU 在零点处是平滑的。

门控线性单元 (GLU) 最初由 Y. N. Dauphin 等人 [19] 定义为：一个经过 sigmoid 的线性变换, 与另一个线性变换逐元素相乘：

$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \quad (6)$$

其中 $\odot$ 表示逐元素乘法。GLU 被认为可以“在保留非线性能力的同时, 为梯度提供一条线性路径, 从而缓解深层架构中的梯度消失问题”。

把 SiLU/Swish 与 GLU 结合起来, 就得到我们将在前馈网络中使用的 SwiGLU：

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2 (\text{SiLU}(W_1 x) \odot W_3 x). \quad (7)$$

其中, $x \in \mathbb{R}^{d_{\text{model}}}$, $W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$; 标准取值为：

$$d_{\text{ff}} = \frac{8}{3}d_{\text{model}}.$$

在实际实现中，为了提高硬件效率，可以把它舍入到附近的 64 的倍数。

N. Shazeer [20] 首先提出把 SiLU/Swish 激活与 GLU 结合，并通过实验表明：在语言建模任务中，SwiGLU 优于 ReLU 和不带门控的 SiLU 等基线。你将在本作业后面比较 SwiGLU 与 SiLU。尽管我们已经为这些组件给出一些启发式解释，相关论文也提供了更多证据，但仍应保持经验主义视角。Shazeer 论文中有一句如今很有名的话：

“我们无法解释这些架构为何似乎有效；与其他一切一样，我们把它们的成功归因于神明的恩惠。”

问题 (`positionwise_feedforward`) : 实现逐位置前馈网络 (2 分)

提交内容：实现由 SiLU 激活函数和 GLU 组成的 SwiGLU 前馈网络。

注意：在这个特定问题中，为了数值稳定性，你可以直接在实现中使用 `torch.sigmoid`。

实现中应把 `d_ff` 设置为大约 $(8)/(3)d_{\text{model}}$ ，同时确保内部前馈层维度是 64 的倍数，以充分利用硬件。

若要使用课程提供的测试检查实现，需要实现测试适配器 `[adapters.run_swiglu]`，然后运行：

```
uv run pytest -k test_swiglu
```

3.4.3 相对位置 embedding

为了向模型注入位置信息，我们将实现旋转位置 embedding (Rotary Position Embeddings; J. Su 等人 [2021])，通常简称 RoPE。对于 token 位置 i 上的 query token：

$$q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d,$$

我们应用一个成对旋转矩阵 R^i ，得到：

$$q'^{(i)} = R^i q^{(i)} = R^i W_q x^{(i)}.$$

这里， R^i 把 embedding 元素对 $q^{(i)}_{2k-1:2k}$ 当作二维向量，并旋转角度：

$$\theta_{i,k} = i\Theta^{(2k-2)/d}, \quad k \in \{1, \dots, d/2\},$$

其中 Θ 是某个常数。因此，可以把 R^i 看作一个大小为 $d \times d$ 的分块对角矩阵；对 $k \in \{1, \dots, d/2\}$ ，其第 k 个块为：

$$R_k^i = \begin{pmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix}. \quad (8)$$

完整旋转矩阵为：

$$R^i = \begin{pmatrix} R_1^i & 0 & 0 & \cdots & 0 \\ 0 & R_2^i & 0 & \cdots & 0 \\ 0 & 0 & R_3^i & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R_{d/2}^i \end{pmatrix}, \quad (9)$$

其中所有 0 都表示 2×2 零矩阵。虽然可以显式构造完整的 $d \times d$ 矩阵，但好的实现应利用该矩阵的结构，更高效地完成变换。

由于我们只关心同一序列内 token 之间的相对旋转，因此可以跨层、跨 batch 复用计算出的 $\cos(\theta_{i,k})$ 与 $\sin(\theta_{i,k})$ 。如需优化，可以让所有层引用同一个 RoPE 模块，并在初始化时通过 `self.register_buffer(persistent=False)` 创建一个预计算的二维 sin/cos buffer，而不是使用 `nn.Parameter`，因为这些固定的余弦和正弦值不需要学习。

随后, 对 $k^{(j)}$ 执行与 $q^{(i)}$ 完全相同的旋转过程, 并使用对应的 R^j 。注意, 该层没有可学习参数。

问题 (rope) : 实现 RoPE (2 分)

提交内容: 实现一个 `RotaryPositionalEmbedding` 类, 把 RoPE 应用于输入张量。建议使用以下接口:

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)
```

构造 RoPE 模块, 并在需要时创建 buffer。

- `theta: float`: RoPE 中的 Θ 值。
- `d_k: int`: query 和 key 向量的维度。
- `max_seq_len: int`: 输入的最大序列长度。
- `device: torch.device | None = None`: 存放 buffer 的设备。

```
def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor
```

处理形状为 $(\dots, \text{seq_len}, d_k)$ 的输入张量, 并返回相同形状的张量。应允许 x 有任意数量的 batch 维度。假设 token 位置是形状为 $(\dots, \text{seq_len})$ 的张量, 用来指定 x 在序列维上的 token 位置。

应使用 token 位置在序列维上索引你的 cos 与 sin 张量; 这些张量可以是预计算的。

若要测试, 补全 `[adapters.run_rope]`, 并确保能通过:

```
uv run pytest -k test_rope
```

3.4.4 Scaled Dot-Product Attention

现在, 我们实现 A. Vaswani 等人 [8] 第 3.2.1 节描述的 scaled dot-product attention。作为准备, Attention 操作的定义会使用 softmax。Softmax 接收一个未归一化的分数向量, 把它转换为归一化分布:

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^n \exp(v_j)}. \quad (10)$$

当 v_i 很大时, $\exp(v_i)$ 可能变成 `inf`, 随后 `inf/inf = NaN`。注意到 softmax 对“给所有输入同时加上任意常数 c ”保持不变, 我们可以利用这一性质提升数值稳定性: 通常从 v 的每个元素中减去 v 的最大值, 使新的最大元素变成 0。现在, 你要利用这一技巧实现 softmax。

问题 (softmax) : 实现 softmax (1 分)

提交内容: 编写一个对张量应用 softmax 的函数。该函数接收两个参数: 一个张量和一个维度 i ; 它应沿输入张量第 i 个维度应用 softmax。输出张量应与输入张量形状相同, 但第 i 个维度上的值应形成归一化概率分布。

为避免数值稳定性问题, 请从第 i 个维度的所有元素中减去该维度的最大值。

若要测试, 补全 `[adapters.run_softmax]`, 并确保能通过:

```
uv run pytest -k test_softmax_matches_pytorch
```

现在可以用数学形式定义 Attention 操作:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \quad (11)$$

其中, $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, $V \in \mathbb{R}^{m \times d_v}$ 。这里的 Q 、 K 和 V 都是该操作的输入; 注意, 它们并不是可学习参数。

Masking: 有时需要对注意力操作的输出施加 mask。Mask 的形状应为 $M \in \{\text{True}, \text{False}\}^{n \times m}$ 。这个布尔矩阵的第 i 行表示 query i 应关注哪些 key。按照惯例——虽然略显反直觉——位置 (i, j) 上的 `True` 表示 query i 会关注 key j ; `False` 表示 query 不会关注该 key。换言之, 在值为 `True` 的 (i, j) 对上, “信息可以流动”。

例如, 一个 1×3 mask 矩阵 `[[True, True, False]]` 表示唯一的 query 向量只关注前两个 key。与分别对子序列计算注意力相比, 使用 mask 在计算上高效得多。具体做法是: 在 softmax 之前的值 $QK^T/\sqrt{d_k}$ 中, 对 mask 矩阵值为 `False` 的每个位置加上 $-\infty$ 。

问题 (scaled_dot_product_attention) : 实现 scaled dot-product attention (5 分)

提交内容：实现 scaled dot-product attention 函数。实现应能处理形状为 $(batch_size, \dots, seq_len, d_k)$ 的 key 和 query，以及形状为 $(batch_size, \dots, seq_len, d_v)$ 的 value；其中 $\dots$ 表示可能存在的任意数量的其他类 batch 维度。返回输出形状应为 $(batch_size, \dots, seq_len, d_v)$ 。有关类 batch 维度的讨论见第 3.2 节。

实现还应支持一个可选的、由用户提供的布尔 mask，形状为 (seq_len, seq_len) 。Mask 值为 True 的位置所对应的注意力概率总和应为 1；mask 值为 False 的位置所对应的注意力概率应为 0。

若要使用课程提供的测试检查实现，需要实现测试适配器 `[adapters.run_scaled_dot_product_attention]`。以下命令在三阶输入张量上测试：

```
uv run pytest -k test_scaled_dot_product_attention
```

以下命令在四阶输入张量上测试：

```
uv run pytest -k test_4d_scaled_dot_product_attention
```

3.4.5 因果多头自注意力

我们将实现 A. Vaswani 等人 [8] 第 3.2.2 节描述的多头自注意力。回顾其数学定义：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h), \quad (12)$$

其中：

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i). \quad (13)$$

Q_i 、 K_i 和 V_i 分别是 Q 、 K 、 V 的 embedding 维度中第 $i \in \{1, \dots, h\}$ 个大小为 d_k 或 d_v 的切片。这里的 Attention 是第 3.4.4 节定义的 scaled dot-product attention。由此得到多头自注意力操作：

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x). \quad (14)$$

可学习参数为：

- $W_Q \in \mathbb{R}^{hd_k \times d_{\text{model}}}$;
- $W_K \in \mathbb{R}^{hd_k \times d_{\text{model}}}$;
- $W_V \in \mathbb{R}^{hd_v \times d_{\text{model}}}$;
- $W_O \in \mathbb{R}^{d_{\text{model}} \times hd_v}$ 。

由于多头注意力会对 Q 、 K 、 V 进行切片，可以把 W_Q 、 W_K 、 W_V 理解为在输出维度上按注意力头分开。实现正确后，你应当总共只用三次矩阵乘法完成 key、value 和 query 投影。

因果 mask

实现必须防止模型关注序列中的未来 token。换言之，假设模型接收 token 序列 $t_1, \dots, t_n$ ，我们要为前缀 $t_1, \dots, t_i$ （其中 $i < n$ ）计算下一个词预测，则模型不能访问或关注位置 $t_{i+1}, \dots, t_n$ 的 token 表示。这是因为推理生成文本时模型无法访问这些 token；而且未来 token 会泄露真实下一个词的身份，使语言模型预训练目标变得毫无难度。

对于输入序列 $t_1, \dots, t_n$ ，一种朴素做法是针对序列的 n 个不同前缀分别运行一次多头自注意力，共运行 n 次，从而阻止访问未来 token。这里改用因果注意力 mask，使 token i 只能关注序列中满足 $j \leq i$ 的位置。可以用 `torch.triu` 或广播后的索引比较构造该 mask；同时应利用第 3.4.4 节的 scaled dot-product attention 已经支持注意力 masking 这一事实。

应用 RoPE

RoPE 应用于 query 和 key 向量，但不应用于 value 向量。此外，head 维度应当视为 batch 维度，因为在多头注意力中，各个 head 独立执行注意力。这意味着，应对每个 head 的 query 和 key 向量应用完全相同的 RoPE 旋转。

进阶目标：尝试把 key、query 和 value 投影合并到一个权重矩阵中，从而只需一次矩阵乘法。

问题 (multihead_self_attention) : 实现因果多头自注意力 (5 分)

提交内容：把因果多头自注意力实现为一个 `torch.nn.Module`。实现至少应接受：

- `d_model: int` : Transformer block 输入的维度。
- `num_heads: int` : 多头自注意力使用的 head 数量。

遵循 A. Vaswani 等人 [8], 令：

$$d_k = d_v = \frac{d_{\text{model}}}{h}.$$

若要使用课程提供的测试检查实现，请实现测试适配器 `[adapters.run_multihead_self_attention]`，然后运行：

```
uv run pytest -k test_multihead_self_attention
```

3.5 完整的 Transformer 语言模型

首先组装 Transformer block；此处可回看原文图 2。一个 Transformer block 包含两个“子层”：一个用于多头自注意力，另一个用于 SwiGLU 前馈网络。每个子层都先执行 RMSNorm，再执行主要操作（MHA/FF），最后加上残差连接。

具体而言，Transformer block 的前半部分，也就是第一个“子层”，应通过以下更新从输入 x 得到输出 y ：

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \quad (15)$$

问题 (transformer_block) : 实现 Transformer block (3 分)

按第 3.4 节的描述和原文图 2 所示，实现 pre-norm Transformer block。你的 Transformer block 至少应接受：

- `d_model: int` : Transformer block 输入的维度。
- `num_heads: int` : 多头自注意力使用的 head 数量。
- `d_ff: int` : 逐位置前馈网络内部层的维度。

若要测试，实现适配器 `[adapters.run_transformer_block]`，然后运行：

```
uv run pytest -k test_transformer_block
```

提交内容：能通过课程测试的 Transformer block 代码。

现在，按照原文图 1 的高层结构把各个 block 组合起来。遵循第 3.1.0.1 节对 embedding 的描述，让 embedding 输出经过 `num_layers` 个 Transformer block，然后再经过最终层归一化和 LM head，得到词表上的未归一化分布，即 logits。

问题 (transformer_lm) : 实现 Transformer 语言模型 (3 分)

现在把所有组件组装起来。按第 3.1 节及原文图 1 的说明，实现 Transformer 语言模型。你的实现至少应接受前述所有 Transformer block 构造参数，以及以下额外参数：

- `vocab_size: int` : 词表大小，用于确定 token embedding 矩阵的维度。
- `context_length: int` : 最大上下文长度，用于确定 RoPE sin/cos buffer 的维度。
- `num_layers: int` : 使用的 Transformer block 数量。

若要使用课程提供的测试检查实现，首先需要实现测试适配器 `[adapters.run_transformer_lm]`，然后运行：

```
uv run pytest -k test_transformer_lm
```

提交内容：一个能通过上述测试的 Transformer LM 模块。

资源核算

理解 Transformer 各部分如何消耗算力和内存非常有用。下面进行基础的“FLOPs 核算”。Transformer 中绝大多数 FLOPs 来自矩阵乘法，因此核心方法很简单：

1. 列出 Transformer 一次前向传播中的全部矩阵乘法。

2. 把每次矩阵乘法转换为所需的 FLOPs。

第二步会用到以下事实：

规则：给定 $A \in \mathbb{R}^{m \times n}$ 与 $B \in \mathbb{R}^{n \times p}$ ，矩阵乘积 AB 需要 $2mnp$ FLOPs。

原因是 $(AB)[i,j] = A[i,:]\cdot B[:,j]$ ，该点积需要 n 次加法和 n 次乘法，共 $2n$ FLOPs。又因为矩阵乘积 AB 有 $m \times p$ 个元素，所以总 FLOPs 为 $(2n)(mp) = 2mnp$ 。

在完成下一题前，最好逐一检查 Transformer block 和 Transformer LM 的各个组件，列出全部矩阵乘法及相应 FLOPs 成本。

问题 (transformer_accounting) : Transformer LM 资源核算 (5 分)

(a) 考虑一个采用本作业架构、规模与 GPT-2 XL 相当的模型，配置如下：

```
vocab_size: 50,257
context_length: 1,024
num_layers: 48
d_model: 1,600
num_heads: 25
d_ff: 4,288
```

其中 d_{ff} 是最接近 $(8/3) \times 1,600$ 的 64 的倍数。

假设用该配置构建模型，它共有多少可训练参数？假设每个参数都用单精度浮点数表示，仅加载该模型需要多少内存？

提交内容：用一到两句话回答。

(b) 找出这个 GPT-2 XL 形状模型完成一次前向传播所需的矩阵乘法。这些矩阵乘法总共需要多少 FLOPs？假设输入序列包含 $context_length$ 个 token。

提交内容：列出全部矩阵乘法及其说明，并给出所需 FLOPs 总数。

(c) 根据以上分析，模型的哪些部分需要最多 FLOPs？

提交内容：用一到两句话回答。

(d) 对 GPT-2 small (12 层、 $d_{model}=768$ 、12 个 head)、GPT-2 medium (24 层、 $d_{model}=1024$ 、16 个 head) 和 GPT-2 large (36 层、 $d_{model}=1280$ 、20 个 head) 重复上述分析。随着模型增大，Transformer LM 的哪些部分在总 FLOPs 中所占比例变大或变小？

提交内容：对每个模型给出各模型组件及其 FLOPs 明细，并把它表示为一次前向传播总 FLOPs 的比例；另外，用一到两句话说明模型大小变化如何影响各组件的 FLOPs 比例。

(e) 取 GPT-2 XL，并把上下文长度增大到 16,384。一次前向传播的总 FLOPs 如何变化？各模型组件对 FLOPs 的相对贡献如何变化？

提交内容：用一到两句话回答。

4 训练 Transformer 语言模型

现在，我们已经有了预处理数据的步骤（通过分词器）以及模型（Transformer）。剩下的是构建支持训练的全部代码，包括：

- 损失：定义损失函数（交叉熵）。
- 优化器：定义用于最小化该损失的优化器（AdamW）。
- 训练循环：构建加载数据、保存 checkpoint 和管理训练的全部配套基础设施。

4.1 交叉熵损失

回顾一下：对每个长度为 $m+1$ 的序列 x ，以及每个 $i=1, \dots, m$ ，Transformer 语言模型都定义了一个分布：

$$p_{\theta}(x_{i+1} \mid x_{1:i}).$$

给定由长度为 $m+1$ 的序列组成的训练集 D ，标准交叉熵（负对数似然）损失函数定义为：

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^m -\log p_{\theta}(x_{i+1} \mid x_{1:i}). \quad (16)$$

注意，Transformer 的一次前向传播会同时产生所有 $i=1, \dots, m$ 对应的 $p_{\theta}(x_{i+1} \mid x_{1:i})$ 。

具体而言，Transformer 在每个位置 i 计算 logits $\mathbf{o}_i \in \mathbb{R}^{\text{vocab_size}}$ ，于是：

$$p(x_{i+1} \mid x_{1:i}) = \text{softmax}(\mathbf{o}_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab_size}} \exp(o_i[a])}. \quad (17)$$

通常，交叉熵损失是相对于 logits 向量 $\mathbf{o}_i \in \mathbb{R}^{\text{vocab_size}}$ 和目标 x_{i+1} 定义的。

$\mathbf{o}_i[k]$ 表示向量 $\mathbf{o}_i$ 在索引 k 处的值。

这对应于集中在 x_{i+1} 上的 Dirac delta 分布与预测分布 $\text{softmax}(\mathbf{o}_i)$ 之间的交叉熵。

与 softmax 一样，实现交叉熵损失时也必须谨慎处理数值问题。

问题 (cross_entropy) : 实现交叉熵 (1 分)

提交内容：编写一个计算交叉熵损失的函数。函数接收预测 logits ($\mathbf{o}_i$) 和目标 (x_{i+1})，并计算：

$$\ell_i = -\log \text{softmax}(\mathbf{o}_i)[x_{i+1}].$$

你的函数应做到：

- 减去最大元素，以提高数值稳定性。
- 只要可能，就约去 $\log$ 和 $\exp$ 。
- 处理任意额外 batch 维度，并返回整个 batch 上的平均值。与第 3.2 节一样，我们假设类 batch 维度始终位于词表大小维之前。

实现 `[adapters.run_cross_entropy]`，然后运行：

```
uv run pytest -k test_cross_entropy
```

困惑度

交叉熵足以用于训练，但评估模型时还需要报告困惑度。对于长度为 m 的序列，若交叉熵损失为 $\ell_1, \dots, \ell_m$ ，则：

$$\text{perplexity} = \exp \left(\frac{1}{m} \sum_{i=1}^m \ell_i \right). \quad (18)$$

4.2 SGD 优化器

现在已经有了损失函数，接下来开始探索优化器。最简单的基于梯度的优化器是随机梯度下降 (Stochastic Gradient Descent, SGD)。从随机初始化参数 θ_0 开始；随后，对每一步 $t=0, \dots, T-1$ ，执行更新：

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \quad (19)$$

其中， B_t 是从数据集 D 中随机抽取的一批数据；学习率 α_t 和 batch 大小 $|B_t|$ 都是超参数。

4.2.1 在 PyTorch 中实现 SGD

为了实现自己的优化器，我们将继承 PyTorch 的 `torch.optim.Optimizer` 类。一个 `Optimizer` 子类必须实现两个方法：

`def __init__(self, params, ...)`：初始化优化器。这里的 `params` 是需要优化的一组参数；如果用户希望对模型不同部分使用不同的学习率等超参数，也可以传入参数组。务必把 `params` 传给基类的 `__init__` 方法；基类会存储这些参数，供 `step` 使用。你还可以根据具体优化器接收额外参数（学习率就是常见例子），并以字典形式传给基类构造函数；字典的 key 是你为这些参数选择的名称字符串。

`def step(self)`：执行一次参数更新。训练循环会在反向传播之后调用它，因此此时可以访问上一个 batch 的梯度。该方法应遍历每个参数张量 `p`，并根据 `p.grad`（若存在）原地修改参数，即设置 `p.data`。`p.data` 保存与该参数关联的张量，`p.grad` 则是损失相对于该参数的梯度张量。

PyTorch 优化器 API 有一些细节，通过例子说明更容易理解。为了使示例更丰富，我们实现 SGD 的一个轻微变体：学习率在训练过程中逐渐衰减，从初始学习率 α 开始，步长随时间不断减小：

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t). \quad (20)$$

下面是该 SGD 版本作为 PyTorch `Optimizer` 的实现方式：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"] # 获取学习率。
            for p in group["params"]:
                if p.grad is None:
                    continue

                state = self.state[p] # 获取与 p 关联的状态。
                t = state.get("t", 0) # 从状态中读取迭代次数；若无则取 0。
                grad = p.grad.data # 获取损失相对于 p 的梯度。
                p.data -= lr / math.sqrt(t + 1) * grad # 原地更新权重张量。
                state["t"] = t + 1 # 迭代次数加一。
```

```
return loss
```

在 `__init__` 中，我们把参数和默认超参数一起传给基类构造函数；参数可能被分组，每组可以有不同超参数。如果参数只是一个 `torch.nn.Parameter` 对象集合，基类构造函数会创建一个参数组，并为它分配默认超参数。

在 `step` 中，我们依次遍历每个参数组、再遍历该组中的每个参数，并应用公式 (20)。这里把迭代次数保存为与每个参数关联的状态：先读取该值，在梯度更新中使用它，然后再更新该值。

API 规定用户可能传入一个可调用的 `closure`，用于在优化器 `step` 前重新计算损失。本作业所用优化器不需要这一功能，但为遵守 API，我们仍将其加入。

可以使用以下最小训练循环示例观察它如何工作：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad() # 重置所有可学习参数的梯度。
    loss = (weights**2).mean() # 计算标量损失。
    print(loss.cpu().item())
    loss.backward() # 运行反向传播，计算梯度。
    opt.step() # 执行优化器更新。
```

这就是训练循环的典型结构：每次迭代中，计算损失，然后让优化器执行一步更新。训练语言模型时，可学习参数来自模型；在 PyTorch 中，`m.parameters()` 会返回该参数集合。损失会在抽样得到的一批数据上计算，但训练循环的基本结构不变。

问题 (learning_rate_tuning) : 调整学习率 (1 分)

我们将看到，学习率是对训练影响最大的超参数之一。现在用上面的玩具示例实际观察这一点。把学习率分别改为 `1e1`、`1e2` 和 `1e3`，每种只运行 10 次训练迭代。每个学习率下的损失会发生什么？它下降得更快、更慢，还是发散，也就是在训练期间增大？

提交内容：用一到两句话说明你观察到的行为。

4.3 AdamW

现代语言模型通常不使用 SGD，而是使用更复杂的优化器进行训练。近年使用的大多数优化器都是 Adam 优化器 [D. P. Kingma et al., 2015] 的衍生版本。本作业使用 AdamW [I. Loshchilov et al., 2019]；它在近期研究中应用广泛。AdamW 对 Adam 进行了修改：以与梯度更新解耦的方式加入权重衰减（每次迭代都把参数向 0 拉近），从而改善正则化。我们将按 I. Loshchilov 等人 [23] 的算法 2 实现 AdamW。

AdamW 是有状态的：对每个参数，它都会维护一阶矩和二阶矩的运行估计。因此，AdamW 用额外内存换取更好的稳定性和收敛性。

除学习率 α 外，AdamW 还有一对控制矩估计更新的超参数 (β_1, β_2) ，以及权重衰减率 λ 。典型应用把 (β_1, β_2) 设为 $(0.9, 0.999)$ ；但 LLaMA [H. Touvron et al., 2023] 和 GPT-3 [T. B. Brown et al., 2020] 等大语言模型常使用 $(0.9, 0.95)$ 。算法如下，其中 ϵ 是一个很小的数（例如 10^{-8} ），用于在 v 极小时改善数值稳定性。

算法 1：AdamW 优化器

1. `init( $\theta$ )`：初始化可学习参数。
2. `$m \leftarrow 0$` ：一阶矩向量的初始值，形状与 θ 相同。
3. `$v \leftarrow 0$` ：二阶矩向量的初始值，形状与 θ 相同。
4. 对 `$t=1, \dots, T$` ：
 1. 抽取一批数据 `$B_t$` 。
 2. `$g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$` ：计算损失梯度。
 3. `$\alpha_t \leftarrow \alpha(v(1-\beta_2^t))/(1-\beta_1^t)$` ：计算第 t 次迭代的修正学习率。

4. $\theta \leftarrow \theta - \alpha \lambda \theta$: 应用权重衰减。
5. $m \leftarrow \beta_1 m + (1 - \beta_1) g$: 更新一阶矩估计。
6. $v \leftarrow \beta_2 v + (1 - \beta_2) g^2$: 更新二阶矩估计。
7. $\theta \leftarrow \theta - \alpha_t(m) / (\sqrt{v} + \epsilon)$: 应用经过矩估计修正的权重更新。

注意, t 从 1 开始。现在, 你需要实现该优化器。

问题 (adamw) : 实现 AdamW (2 分)

提交内容: 把 AdamW 优化器实现为 `torch.optim.Optimizer` 的子类。你的类应在 `__init__` 中接收学习率 α , 以及 β 、 ϵ 和 λ 超参数。

为帮助保存状态, `Optimizer` 基类提供字典 `self.state`, 它把 `nn.Parameter` 对象映射到另一个字典; 后者可以保存该参数所需的任何信息。对 AdamW 而言, 这些信息就是矩估计。

实现 `[adapters.get_adamw_cls]`, 并确保能通过:

```
uv run pytest -k test_adamw
```

问题 (adamw_accounting) : AdamW 训练的资源核算 (2 分)

下面计算运行 AdamW 所需的内存和计算量。假设每个张量都使用 `float32`。

(a) 运行 AdamW 所需的峰值内存是多少? 请把答案分解为参数、激活值、梯度和优化器状态所占的内存。用 `batch_size` 和模型超参数 (`vocab_size`、`context_length`、`num_layers`、`d_model`、`num_heads`) 表示答案。假设 $d_{ff} = (8/3)d_{model}$ 。

为简化激活值内存计算, 只考虑以下组件:

- Transformer block :
 - RMSNorm ;
 - 多头自注意力子层: QKV 投影、 QK^T 矩阵乘法、softmax、value 加权求和、输出投影;
 - 逐位置前馈网络 (SwiGLU) : W_1 、 W_2 、门控分支上的 SiLU、逐元素乘积、 W_3 。
- 最终 RMSNorm ;
- 输出 embedding ;
- logits 上的交叉熵。

提交内容: 分别给出参数、激活值、梯度和优化器状态的代数表达式, 以及总量表达式。

(b) 把答案实例化为 GPT-2 XL 形状模型, 得到一个只依赖 `batch_size` 的表达式。在 80 GB 内存限制下, 最大可使用多大的 batch?

提交内容: 一个形如 $a \cdot \text{batch_size} + b$ 的表达式, 其中 a, b 是数值; 并给出最大 batch 大小。

(c) 运行 AdamW 的一步更新需要多少 FLOPs?

提交内容: 给出代数表达式, 并简要说明理由。

(d) 模型 FLOPs 利用率 (Model FLOPs Utilization, MFU) 定义为: 观测吞吐量 (token/秒) 与硬件理论峰值 FLOP 吞吐量之比 [A. Chowdhery et al., 2022]。NVIDIA H100 GPU 对“float32” (实际是 TensorFloat-32, 原文称其现实中为“bfloat19”) 运算的理论峰值为 495 teraFLOP/s。假设能达到 50% MFU, 在单张 H100 上, 以 batch 大小 1024 训练 GPT-2 XL 400K 步需要多长时间? 遵循 J. Kaplan 等人 [25] 和 J. Hoffmann 等人 [26], 假设反向传播的 FLOPs 是前向传播的两倍。

提交内容: 给出训练所需小时数, 并简要说明理由。

4.4 学习率调度

在训练过程中, 使损失下降最快的学习率往往会发生变化。训练 Transformer 时, 通常使用学习率调度: 开始时采用较大的学习率, 以便快速更新; 随着训练进行, 再逐渐衰减到较小值。 本作业将实现训练 LLaMA [H. Touvron et al., 2023] 所使用的余弦退火调度。

调度器本质上只是一个函数：它接收当前步数 t 和其他相关参数（例如初始与最终学习率），返回第 t 步梯度更新应使用的学习率。最简单的是常数调度：无论输入什么 t ，都返回相同学习率。

余弦退火学习率调度接收：(i) 当前迭代 t ；(ii) 最大学习率 $\alpha_{\max}$ ；(iii) 最小或最终学习率 $\alpha_{\min}$ ；(iv) warm-up 迭代次数 T_w ；(v) 余弦退火结束迭代 T_c 。第 t 次迭代的学习率定义为：

Warm-up：若 $t < T_w$ ，则

$$\alpha_t = \frac{t}{T_w} \alpha_{\max}.$$

余弦退火：若 $T_w \leq t \leq T_c$ ，则

$$\alpha_t = \alpha_{\min} + \frac{1}{2} \left(1 + \cos \left(\frac{t - T_w}{T_c - T_w} \pi \right) \right) (\alpha_{\max} - \alpha_{\min}).$$

退火后：若 $t > T_c$ ，则

$$\alpha_t = \alpha_{\min}.$$

有时也会使用让学习率重新升高（restart）的调度，以帮助跳出局部极小值。

问题（learning_rate_schedule）：实现带 warm-up 的余弦学习率调度（1 分）

编写一个函数，接收 t 、 $\alpha_{\max}$ 、 $\alpha_{\min}$ 、 T_w 和 T_c ，并按上面的调度返回学习率 α_t 。然后实现 `[adapters.get_lr_cosine_schedule]`，并确保能通过：

```
uv run pytest -k test_get_lr_cosine_schedule
```

4.5 梯度裁剪

训练期间，有时会遇到产生巨大梯度的训练样本，这会使训练不稳定。实践中常用梯度裁剪（gradient clipping）缓解这一问题。其思想是：每次反向传播后、执行优化器 step 前，对梯度范数设置上限。

给定所有参数的梯度 g ，计算其 ℓ_2 范数 $\|g\|_2$ 。如果该范数小于最大值 M ，则保持 g 不变；否则，按因子 $(M)/(\|g\|_2 + \epsilon)$ 缩小 g ，其中加入一个很小的 ϵ （例如 10^{-6} ）以保证数值稳定性。注意，处理后的范数会略小于 M 。

问题（gradient_clipping）：实现梯度裁剪（1 分）

编写一个实现梯度裁剪的函数。函数应接收一个参数列表和最大 ℓ_2 范数，并原地修改每个参数的梯度。使用 $\epsilon=10^{-6}$ ，即 PyTorch 默认值。然后实现适配器 `[adapters.run_gradient_clipping]`，并确保能通过：

```
uv run pytest -k test_gradient_clipping
```

5 训练循环

现在，终于可以把目前构建的主要组件组合起来：完成分词的数据、模型和优化器。

5.1 数据加载器

分词后的数据（例如你在 `tokenizer_experiments` 中准备的数据）是一个 token 序列：

$$x = (x_1, \dots, x_n).$$

即使源数据由多个独立文档组成（例如不同网页或源代码文件），常见做法仍是把它们全部拼接为一个 token 序列，并在文档之间添加分隔符，例如 `<|endoftext|>` token。

数据加载器把该序列转换为 batch 流。每个 batch 由 B 个长度为 m 的序列组成，并与对应的、同样长度为 m 的下一个 token 目标配对。例如，当 $B=1, m=3$ 时， $([x_2, x_3, x_4], [x_3, x_4, x_5])$ 就可能是一个 batch。

这种数据加载方式从多个方面简化了训练。首先，任何 $1 \leq i \leq n-m$ 都能产生有效训练序列，因此抽取训练序列非常简单。其次，所有训练序列长度相同，无需对输入进行 padding；这能提高硬件利用率，也有助于增大 batch 大小 B 。最后，抽样训练数据时无需把完整数据集加载进内存，因此很容易处理原本无法装入内存的大型数据集。

问题 (data_loading) : 实现数据加载 (2 分)

提交内容：编写一个函数，接收：

- NumPy 数组 x ，即包含 token ID 的整数数组；
- `batch_size`；
- `context_length`；
- PyTorch 设备字符串，例如 `'cpu'` 或 `'cuda:0'`。

函数返回一对张量：抽样得到的输入序列和相应的下一个 token 目标。两个张量的形状都应为 $(batch_size, context_length)$ ，内容为 token ID，并且都应放在指定设备上。

若要使用课程提供的测试检查实现，首先需要实现测试适配器 `[adapters.run_get_batch]`，然后运行：

```
uv run pytest -k test_get_batch
```

低资源提示：在 CPU 或 Apple Silicon 上加载数据

如果计划在 CPU 或 Apple Silicon 上训练语言模型，需要把数据移动到正确设备；之后模型也应使用同一设备。

使用 CPU 时，可以使用设备字符串 `'cpu'`；使用 Apple Silicon (M 系列芯片) 时，可以使用 `'mps'`。

关于 MPS 的更多信息：

- <https://docs.pytorch.org/docs/stable/mps.html>
- <https://docs.pytorch.org/docs/stable/notes/mps.html>
- <https://developer.apple.com/documentation/metalperformanceshaders>

如果数据集大到无法装入内存怎么办？可以使用名为 `mmap` 的 Unix 系统调用。它把磁盘文件映射到虚拟内存，并在访问相应内存位置时才惰性加载文件内容。这样，你可以“假装”整个数据集都在内存中。

NumPy 通过 `np.memmap` 实现该功能。如果原始数组是通过 `np.save` 保存的，也可以把 `mmap_mode='r'` 传给 `np.load`。它们会返回一个类似 NumPy 数组的对象，只在访问条目时按需加载内容。

训练期间从数据集（即 NumPy 数组）抽样时，务必以 memory-mapped 模式加载数据集：根据数组的保存方式，使用 `np.memmap`，或者对 `np.load` 使用 `mmap_mode='r'`。还要确保指定与所加载数组相匹配的 `dtype`。最好显式检查 memory-mapped 数据是否正确，例如确认其中没有超过预期词表大小的值。

5.2 Checkpoint

除加载数据外，训练过程中还需要保存模型。运行训练任务时，我们经常希望能够恢复一个中途停止的训练，例如任务超时、机器故障等。即使一切顺利，也可能希望以后访问中间模型，例如事后研究训练动态、从不同训练阶段的模型中抽样等。

Checkpoint 应包含恢复训练所需的全部状态。至少需要恢复模型权重。使用 AdamW 这样的有状态优化器时，还要保存优化器状态，例如 AdamW 的矩估计。最后，为了恢复学习率调度，还必须知道训练停止时的迭代次数。

PyTorch 可以方便地保存这些内容：每个 `nn.Module` 都有 `state_dict()` 方法，它返回包含全部可学习权重的字典；之后可以用对应的 `load_state_dict()` 恢复权重。任何 `torch.optim.Optimizer` 也是如此。最后，`torch.save(obj, dest)` 可以把对象写入文件路径或类文件对象；该对象可以是包含张量和普通 Python 对象（例如整数）的字典。随后，可以用 `torch.load(src)` 把它重新加载进内存。

问题 (checkpointing) : 实现模型 checkpoint (1 分)

实现以下两个函数，以保存和加载 checkpoint：

```
def save_checkpoint(model, optimizer, iteration, out)
```

该函数应把模型、优化器和迭代次数的全部状态写入类文件对象 `out`。可以调用模型和优化器的 `state_dict` 方法获取相关状态，再使用 `torch.save(obj, out)` 把 `obj` 写入 `out`。PyTorch 在这里既支持路径，也支持类文件对象。典型做法是让 `obj` 成为一个字典，但只要之后可以重新加载 checkpoint，也可以使用其他格式。

函数参数为：

```
model: torch.nn.Module
optimizer: torch.optim.Optimizer
iteration: int
out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
```

```
def load_checkpoint(src, model, optimizer)
```

该函数应从 `src`（路径或类文件对象）加载 checkpoint，并恢复模型与优化器状态。函数应返回 checkpoint 中保存的迭代次数。可以使用 `torch.load(src)` 读取 `save_checkpoint` 保存的内容，再通过模型与优化器各自的 `load_state_dict` 方法把它们恢复到先前状态。

函数参数为：

```
src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
model: torch.nn.Module
optimizer: torch.optim.Optimizer
```

实现 `[adapters.run_save_checkpoint]` 和 `[adapters.run_load_checkpoint]` 适配器，并确保能通过：

```
uv run pytest -k test_checkpointing
```

5.3 训练循环

现在，终于可以在主训练脚本中把你实现的全部组件组合起来。最好让启动不同超参数的训练任务变得方便，例如把超参数作为命令行参数。之后为了研究不同选择对训练的影响，你会多次启动训练任务，这样做很值得。

问题 (training_together) : 组合所有组件 (4 分)

提交内容：编写一个脚本，运行训练循环，在用户提供的输入上训练模型。特别建议训练脚本至少支持：

- 配置并控制各种模型与优化器超参数；
- 使用 `np.memmap` 以内存高效的方式加载大型训练集与验证集；
- 把 checkpoint 序列化到用户提供的路径。
- 定期记录训练与验证性能，例如输出到控制台，和/或记录到 Weights & Biases 等外部服务。

```
wandb.ai
```

6 生成文本

现在我们已经能训练模型，最后还需要让模型能够生成文本。

回顾一下：语言模型接收一个长度为 `sequence_length` 的整数序列（可以带 batch），并产生大小为 `(sequence_length, vocab_size)` 的矩阵。序列中每个位置都对应一个概率分布，用来预测该位置之后的下一个 token。现在，我们将编写若干函数，把这个输出转化为新序列的采样方案。

Softmax

按照标准约定，语言模型的输出是最后一个线性层的输出，即 logits；因此必须使用前面公式 (10) 中的 softmax 操作，把它转换为归一化概率。

解码

为了从模型生成文本（解码），我们先向模型提供一个前缀 token 序列，也就是“prompt”，并让模型产生词表上的概率分布，用于预测序列中的下一个 token。然后，从这个词表分布中抽样，确定下一个输出 token。

具体来说，解码过程的一步接收序列 $x_1 \dots x_t$ ，并通过下式返回 token x_{t+1} ：

$$P(x_{t+1} = i \mid x_{1 \dots t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)}, \quad (21)$$

其中：

$$v = \text{TransformerLM}(x_{1 \dots t})_t \in \mathbb{R}^{\text{vocab_size}}. \quad (22)$$

这里，`TransformerLM` 是我们的模型：它接收长度为 `sequence_length` 的序列，并输出大小为 `(sequence_length, vocab_size)` 的矩阵。我们取该矩阵的最后一个元素，因为要得到第 t 个位置的下一个 token 预测。

由此可以得到一个基础解码器：反复从这些单步条件分布中抽样，并把上一步生成的输出 token 追加到下一解码时间步的输入中，直到生成序列结束 token `<|endoftext|>`，或者达到用户指定的最大生成 token 数。

解码技巧

本作业会试验小型模型，而小型模型有时会生成质量很低的文本。两个简单的解码技巧可以缓解这一问题。

第一种是温度缩放（temperature scaling）。我们使用温度参数 τ 修改 softmax：

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\text{vocab_size}} \exp(v_j/\tau)}. \quad (23)$$

当 $\tau \rightarrow 0$ 时， v 的最大元素会占据主导地位，softmax 输出趋近于集中在该最大元素上的 one-hot 向量。

第二种技巧是 nucleus sampling，也称 top-p sampling。它通过截断低概率 token 来修改抽样分布。设 q 是对大小为 `vocab_size` 的 logits 做温度缩放 softmax 后得到的概率分布。超参数为 p 的 nucleus sampling 按下式产生下一个 token：

$$P(x_{t+1} = i \mid q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j}, & i \in V(p), \\ 0, & \text{otherwise,} \end{cases} \quad (24)$$

其中, $V(p)$ 是满足 $\sum_{j \in V(p)} q_j \geq p$ 的最小索引集合。可以先按概率大小对分布 q 排序, 再从概率最大的词表元素开始选择, 直到累计概率达到目标 p , 从而轻松计算该集合。

问题 (decoding) : 解码 (3 分)

提交内容: 实现一个从语言模型进行解码的函数。建议支持以下功能:

- 为用户提供的 prompt 生成续写, 即接收某个 $x_1 \dots t$, 并持续抽样续写, 直到遇到 `<|endoftext|>` token。
- 允许用户控制最大生成 token 数。
- 给定温度值后, 在抽样前对预测的下一个 token 分布应用 softmax 温度缩放。
- 给定用户指定的阈值, 执行 top-p sampling (A. Holtzman 等人 [2020], 也称 nucleus sampling)。

7 实验

现在，把所有组件组合起来，在预训练数据集上训练（小型）语言模型。

7.1 如何运行实验及提交内容

理解 Transformer 各架构组件设计理由的最好方式，是亲自修改它并运行实验。动手经验无可替代。

因此，能够快速、一致地实验，并记录做过的事情十分重要。为了快速实验，我们会在小规模模型（总参数约 17M）和简单数据集（TinyStories）上运行许多实验。为了保持一致，你将系统地消融组件、改变超参数；为了保留记录，我们会要求你提交实验日志，以及每个实验对应的学习曲线。

为了能够提交损失曲线，请务必定期评估验证损失，并同时记录梯度步数和实际墙钟时间（wall-clock time）。Weights & Biases 等日志基础设施可能会有所帮助。

问题（experiment_log）：实验记录（3 分）

为训练和评估代码创建实验跟踪基础设施，使你能够按照梯度步数与墙钟时间跟踪实验和损失曲线。

提交内容：实验日志基础设施代码，以及本节下列作业问题的实验日志，即记录你尝试过的全部内容的文档。

7.2 TinyStories

我们先从一个非常简单数据集 TinyStories [R. Eldan 等人 [1]] 开始；模型可以在上面快速训练，而且能够观察到一些有趣行为。数据集获取说明见第 1 节。下面是一条数据样例。

示例（tinystories_example）：TinyStories 中的一个样本

从前，有一个名叫 Ben 的小男孩。Ben 喜欢探索周围的世界。他见过许多神奇的东西，例如商店里陈列的漂亮花瓶。一天，Ben 在商店里走着，偶然看到一个非常特别的花瓶。看到它时，Ben 惊呆了！他说：“哇，这真是一个了不起的花瓶！我可以买下它吗？”店主笑着说：“当然可以。你可以把它带回家，给所有朋友看看它有多么神奇！”于是 Ben 把花瓶带回家，并为它感到非常自豪！他把朋友们叫来，向他们展示这个神奇的花瓶。所有朋友都觉得花瓶很漂亮，也不敢相信 Ben 竟然如此幸运。这就是 Ben 如何在商店里找到一个神奇花瓶的故事！

7.2.1 超参数调优

我们会提供一些最基本的起始超参数，并要求你为其余超参数寻找效果良好的设置。

- 词表大小：10,000。典型词表大小从数万到数十万不等。你应改变该参数，观察词表和模型行为如何变化。
- 上下文长度：256。TinyStories 等简单数据集可能不需要很长的序列；但对后面的 OpenWebText 数据，你可能需要改变它。尝试不同取值，并观察它对每次迭代运行时间及最终困惑度的影响。
- `d_model=512`。这略小于许多小型 Transformer 论文所使用的 768 维，但能使实验更快。
- `d_ff=1344`。这约等于 $(8)/(3)d_{\text{model}}$ ，同时是 64 的倍数，有利于 GPU 性能。
- RoPE theta 参数： $\Theta=10000$ 。
- 层数和 head 数：4 层、16 个 head。二者结合后，模型约有 17M 个不含 embedding 的参数，是一个相当小的 Transformer。
- 处理的 token 总数：327,680,000。你的 `batch_size × 总步数 × context_length` 应大约等于该值。

你需要通过一些试错，为下列其他超参数找到良好的默认设置：学习率、学习率 warm-up、AdamW 的其他超参数（ $\beta_1, \beta_2, \epsilon$ ）以及权重衰减。D. P. Kingma 等人 [22] 中可以找到这些超参数的一些典型选择。

7.2.2 组合所有组件

现在，可以把所有内容组合起来：获得训练好的 BPE 分词器，对训练集进行分词，并把结果输入你编写的训练循环。

重要说明：如果实现正确且高效，使用上述超参数在一张 B200 GPU 上的运行时间应约为 20–30 分钟。如果耗时长得多，请检查数据加载、checkpoint 或验证损失代码是否成为运行瓶颈，同时确认实现已经正确批量化。

7.2.3 调试模型架构的提示与技巧

我们强烈建议熟悉 IDE 内置调试器，例如 VSCode/Zed。与使用 `print` 语句调试相比，它能节省时间。如果使用文本编辑器，可以使用 `ipdb` 等工具。调试模型架构时，其他良好实践包括：

- 开发任何神经网络架构时，常见的第一步是让模型对单个 minibatch 过拟合。如果实现正确，应能迅速把训练损失压到接近 0。
- 在不同模型组件中设置调试断点，检查中间张量形状，确保它们符合预期。
- 监控激活值、模型权重和梯度的范数，确保它们没有爆炸或消失。

问题 (learning_rate) : 调优学习率 (2 B200 小时) (3 分)

学习率是最重要的超参数之一。以你训练的基础模型为起点，回答以下问题：

(a) 对多个学习率执行超参数扫描，并报告最终损失；若优化器发散，则注明发散。

提交内容：多个学习率对应的学习曲线，并解释你的超参数搜索策略。

提交内容：一个在 TinyStories 上每 token 验证损失不超过 1.45 的模型。

低资源提示：在 CPU 或 Apple Silicon 上训练少量步数

如果使用 `cpu` 或 `mps`，应把处理 token 总数降到 40,000,000；这足以产生较为流畅的文本。也可以把目标验证损失从 1.45 放宽到 2.00。

在配备 36 GB RAM 的 M4 Max 上，使用课程组参考实现和调优后的学习率，我们令：

$\text{batch size} \times \text{总步数} \times \text{context length} = 32 \times 5000 \times 256 = 40,960,000 \text{ tokens}$

CPU 耗时 1 小时 22 分钟，MPS 耗时 36 分钟。在第 5000 步，验证损失达到 1.80。

其他提示：

- 使用 `N` 个训练步时，建议调整余弦学习率衰减，使其正好在第 `N` 步结束衰减，也就是在该步达到最小学习率。
- 使用 MPS 时，不要启用 TF32 kernel，也就是不要像在 CUDA 设备上那样设置：

```
torch.set_float32_matmul_precision('high')
```

我们曾在 MPS (PyTorch 2.9.0) 上尝试启用 TF32 kernel，发现后端有时会悄无声息地使用存在问题的 kernel，导致训练不稳定。

- 可以通过 `torch.compile` 对模型进行 JIT 编译，从而加速训练：

- 在 CPU 上：

```
model = torch.compile(model)
```

- 在 MPS 上，可以通过以下方式在一定程度上优化反向传播：

```
model = torch.compile(model, backend="aot_eager")
```

截至 PyTorch 2.9.0，MPS 不支持使用 Inductor 编译。

(b) 经验上，人们常说最好的学习率位于“稳定性边缘” (edge of stability)。研究学习率开始发散的位置与最佳学习率之间的关系。

提交内容：随学习率增大而得到的多条学习曲线，其中至少包含一次发散运行；并分析它与收敛速度之间的关系。

接下来改变 batch 大小，观察训练如何变化。Batch 大小很重要：更大的矩阵乘法能使 GPU 效率更高；但 batch 是否总是越大越好？下面通过实验寻找答案。

问题 (batch_size_experiment) : 改变 batch 大小 (1 B200 小时) (1 分)

从 1 一直改变到 GPU 内存所允许的最大值。中间至少尝试若干个 batch 大小，包括 64、128 等典型取值。

提交内容：不同 batch 大小运行对应的学习曲线；必要时重新优化学习率。

提交内容：用几句话讨论你对 batch 大小及其训练影响的发现。

现在已经有了解码器，可以生成文本并观察模型效果。作为参考，你得到的输出至少应与下面示例一样好。

示例 (ts_generate_example) : TinyStories 语言模型的生成样本

从前，有一个名叫 Lily 的漂亮女孩。她喜欢吃口香糖，尤其喜欢那块大的黑色口香糖。一天，Lily 的妈妈请她帮忙做晚饭。Lily 非常兴奋！她喜欢帮助妈妈。Lily 的妈妈晚饭做了一大锅汤。Lily 很开心，说：“谢谢你，妈妈！我爱你。”她帮妈妈把汤倒进一个大碗。晚饭后，Lily 的妈妈又做了一些美味的汤。Lily 很喜欢！她说：“谢谢你，妈妈！这汤太好喝了！”妈妈笑着说：“你喜欢我就高兴，Lily。”她们做完了饭，又继续一起烹饪。故事结束。

低资源提示：在 CPU 或 Apple Silicon 上生成文本

如果你使用的是只处理 40M token 的低资源配置，生成结果仍应看起来像英语，但不会像上面的示例那样流畅。下面是课程组在 40M token 上训练的 TinyStories 语言模型生成的样本：

从前，有一个名叫 Sue 的小女孩。Sue 有一颗她非常喜欢的牙齿。那是他的最好脑袋。一天，Sue 出去散步，遇到了一只瓢虫！她们成了好朋友，一起在小路上玩耍。

“嘿，Polly！我们出去吧！”Tim 说道。Sue 看着天空，发现很难找到一种闪闪发光的舞蹈方式。她微笑着，同意去帮助那段说话！

当 Sue 看着天空移动时，那是什么。她

下面是准确的问题陈述及提交要求：

问题 (generate) : 生成文本 (1 分)

使用你的解码器和训练好的 checkpoint，报告模型生成的文本。可能需要调整解码参数 (temperature、top-p 等) 才能获得流畅输出。

提交内容：至少 256 个 token 的文本输出；如果更早遇到第一个 `<|endoftext|>` token，则输出到该位置为止。另请简要评论输出的流畅度，并指出至少两个影响输出好坏的因素。

7.3 消融实验与架构修改

真正修改 Transformer 并观察其行为，是理解它的最佳方式。下面进行一些简单消融与修改。

消融 1：层归一化

人们常说层归一化对 Transformer 训练稳定性很重要。但也许我们想冒一点险。现在移除每个 Transformer block 中的 RMSNorm，观察会发生什么。

问题 (layer_norm_ablation) : 移除 RMSNorm 并训练 (0.5 B200 小时) (1 分)

从 Transformer 中移除所有 RMSNorm 并进行训练。在此前的最优学习率下会发生什么？能否通过使用更低学习率获得稳定性？

提交内容：移除 RMSNorm 后的训练学习曲线，以及在最佳学习率下的学习曲线。

提交内容：用几句话评论 RMSNorm 的影响。

接下来研究另一个乍看之下似乎任意的层归一化选择。Pre-norm Transformer block 定义为：

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)), \quad (25)$$

$$y = z + \text{FFN}(\text{RMSNorm}(z)). \quad (26)$$

这是对原始 Transformer 架构少数已经形成“共识”的修改之一。原始架构采用 post-norm：

$$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x)), \quad (27)$$

$$y = \text{RMSNorm}(z + \text{FFN}(z)). \quad (28)$$

现在恢复为 post-norm，观察会发生什么。

问题 (pre_norm_ablation) : 实现 post-norm 并训练 (0.5 B200 小时) (1 分)

把 pre-norm Transformer 实现改成 post-norm。使用 post-norm 模型训练并观察结果。

提交内容：一条 post-norm Transformer 的学习曲线，并与 pre-norm 曲线进行比较。

可以看到，层归一化对 Transformer 行为有重大影响，甚至层归一化所在的位置也很重要。

消融 2 : 位置 embedding

接下来研究位置 embedding 对模型性能的影响。具体而言，我们把基础模型（使用 RoPE）与完全不加入位置 embedding 的模型（NoPE）进行比较。事实证明，我们所实现的带因果 mask 的 decoder-only Transformer，理论上可以在不显式提供位置 embedding 的情况下推断相对或绝对位置信息 [Y.-H. H. Tsai et al., 2019; A. Kazemnejad et al., 2023]。下面通过实验比较 NoPE 与 RoPE 的表现。

问题 (no_pos_emb) : 实现 NoPE (0.5 B200 小时) (1 分)

修改使用 RoPE 的 Transformer 实现，完全移除位置 embedding 信息，并观察结果。

提交内容：一条比较 RoPE 和 NoPE 性能的学习曲线。

消融 3 : SwiGLU 与 SiLU

接下来遵循 N. Shazeer [20] 的工作，测试前馈网络中门控的重要性：比较 SwiGLU 前馈网络与使用 SiLU 激活、但没有门控线性单元（GLU）的前馈网络：

$$\text{FFN}_{\text{SiLU}}(x) = W_2 \text{SiLU}(W_1 x). \quad (29)$$

回顾一下，在 SwiGLU 实现中，我们把内部前馈层维度设为约 $d_{\text{ff}} = (8)/(3)d_{\text{model}}$ ，并保证 $d_{\text{ff} \bmod 64} = 0$ ，以使用 GPU tensor core。在此消融基线中， FFN_{SiLU} 实现应改为 $d_{\text{ff}} = 4d_{\text{model}}$ ，从而使其参数量与默认 SwiGLU 前馈网络大致匹配；后者有三个而不是两个权重矩阵。

问题 (swiglu_ablation) : SwiGLU 与 SiLU (0.5 B200 小时) (1 分)

提交内容：在参数量大致匹配的条件下，比较 SwiGLU 和 SiLU 前馈网络性能的学习曲线。

提交内容：用几句话讨论你的发现。

低资源提示：GPU 资源有限的在线学生应在 TinyStories 上测试修改

在作业余下部分，我们将转向更大规模、噪声更高的网页数据集 OpenWebText，试验架构修改，并可选择向课程排行榜提交结果。

要在 OpenWebText 上把语言模型训练到能流畅生成文本，需要很长时间。因此，建议 GPU 资源有限的在线学生继续在 TinyStories 上测试修改，并用验证损失作为评估性能的指标。

7.4 在 OpenWebText 上运行

现在转向一个由网络爬取内容构建的、更标准的预训练数据集。课程还提供了一个 OpenWebText [A. Gokaslan 等人, 2019] 小样本，形式为单个文本文件；获取方式见第 1 节。

下面是 OpenWebText 中的一个示例。可以看到，文本更加真实、复杂且多样。建议浏览训练数据集，了解网络爬取语料的训练数据是什么样子。

示例 (owt_example) : OWT 中的一个样本

Baseball Prospectus 的技术总监 Harry Pavlidis 在聘用 Jonathan Judge 时冒了一个险。

Pavlidis 知道，正如 Alan Schwarz 在《The Numbers Game》中所写：“美国文化中，没有哪个角落比棒球运动员的表现被统计得更精确、被量化得更热烈。”只需在几个地方点击几下，你就能查到 Noah Syndergaard 的快速球在飞向本垒的过程中每分钟旋转超过 2,100 次，Nelson Cruz 在 2016 年符合资格的击球手中拥有联盟最高的平均击球初速，以及无数

其他仿佛来自电子游戏或科幻小说的琐碎数据。不断上涨的数据海洋，使棒球文化中的一个角色日益重要：业余数据分析爱好者。

这种赋权也带来了更多审视——不仅针对测量本身，也针对测量背后的人与出版物。Pavlidis 对 Baseball Prospectus 很熟悉，深知定量分析稍有瑕疵便会引来反弹。他也知道网站的捕手指标需要重做，而且这项工作需要一个学识渊博的头脑——一个能够处理复杂统计建模问题的人。

“他把我们吓坏了。”——Harry Pavlidis

根据 Judge 的写作，以及两人在一次由网站赞助的棒球场活动中的互动，Pavlidis 直觉认为 Judge “真的懂”。[...]

注意：这次实验可能需要重新调优学习率、batch 大小等超参数。

问题 (main_experiment) : 在 OWT 上实验 (2 B200 小时) (2 分)

使用与 TinyStories 相同的模型架构和总训练迭代次数，在 OpenWebText 上训练语言模型。该模型表现如何？

提交内容：OpenWebText 语言模型的学习曲线。说明其损失与 TinyStories 的差异——应如何解释这些损失？

提交内容：OpenWebText 语言模型生成的文本，格式与 TinyStories 输出相同。文本流畅度如何？为什么在模型和计算预算相同的情况下，输出质量仍比 TinyStories 更差？

7.5 自定义修改与排行榜

恭喜你完成到这里。作业即将结束！现在，你将尝试改进 Transformer 架构，并看看自己的超参数和架构与班上其他学生相比表现如何。

排行榜规则

除下列限制外，没有其他限制：

- 运行时间：提交程序在一张 B200 上最多运行 45 分钟。如果使用 SLURM 或 Modal，可以考虑在提交脚本中主动强制执行该限制。
- 数据：只能使用课程提供的 OpenWebText 训练数据集。

除此之外，你可以自由发挥。

如果需要实现思路，可以参考以下资源：

- 最先进的开源大语言模型家族，例如 Llama 3 [A. Grattafiori et al., 2024] 或 Qwen 2.5 [A. Yang et al., 2024]。
- NanoGPT speedrun 仓库 github.com/KellerJordan/modded-nanogpt。社区成员在其中发布了许多用于“竞速”小规模语言模型预训练的有趣修改。例如，一个可追溯到原始 Transformer 论文的常见修改，是让输入与输出 embedding 共享权重，参见 A. Vaswani 等人 [8] 第 3.4 节，以及 A. Chowdhery 等人 [16] 第 2 节。如果尝试权重共享，可能需要降低 embedding/LM head 初始化的标准差。

在进行完整 45 分钟运行之前，应先在 OpenWebText 的小型子集或 TinyStories 上测试这些修改。

需要提醒的是：某些在这个排行榜中表现良好的修改，不一定能推广到更大规模预训练。课程将在 scaling laws 单元中进一步探讨这一点。

问题 (leaderboard) : 排行榜 (10 B200 小时) (6 分)

按上述排行榜规则训练模型，目标是在 0.75 B200 小时内，使语言模型验证损失尽可能低。

提交内容：最终记录的验证损失；一条明确以墙钟时间为横轴、且横轴小于 45 分钟的学习曲线；以及对所做工作的说明。排行榜提交至少应优于损失为 5.0 的朴素基线。提交地址：

github.com/stanford-cs336/assignment1-basics-leaderboard

参考文献

- [1] R. Eldan and Y. Li, "TinyStories: How Small Can Language Models Be and Still Speak Coherent English?." 2023.
- [2] A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, "OpenWebText corpus." 2019.
- [3] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in Proc. of ACL, 2016.
- [4] C. Wang, K. Cho, and J. Gu, "Neural Machine Translation with Byte-Level Subwords." 2019.
- [5] P. Gage, "A new algorithm for data compression," C Users Journal, vol. 12, no. 2, pp. 23–38, Feb. 1994.
- [6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners." 2019.
- [7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving Language Understanding by Generative Pre-Training." 2018.
- [8] A. Vaswani et al., "Attention is All you Need," in Proc. of NeurIPS, 2017.
- [9] T. Q. Nguyen and J. Salazar, "Transformers without Tears: Improving the Normalization of Self-Attention," in Proc. of IWSWLT, 2019.
- [10] R. Xiong et al., "On Layer Normalization in the Transformer Architecture," in Proc. of ICML, 2020.
- [11] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization." 2016.
- [12] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models." 2023.
- [13] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," in Proc. of NeurIPS, 2019.
- [14] A. Grattafiori et al., "The Llama 3 Herd of Models." [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [15] A. Yang et al., "Qwen2.5 Technical Report," arXiv preprint arXiv:2412.15115, 2024.
- [16] A. Chowdhery et al., "PaLM: Scaling Language Modeling with Pathways." 2022.
- [17] D. Hendrycks and K. Gimpel, "Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units." 2016.
- [18] S. Elfving, E. Uchibe, and K. Doya, "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning." [Online]. Available: <https://arxiv.org/abs/1702.03118>
- [19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, "Language Modeling with Gated Convolutional Networks." [Online]. Available: <https://arxiv.org/abs/1612.08083>
- [20] N. Shazeer, "GLU Variants Improve Transformer." 2020.
- [21] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding." 2021.
- [22] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. of ICLR, 2015.
- [23] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in Proc. of ICLR, 2019.
- [24] T. B. Brown et al., "Language Models are Few-Shot Learners," in Proc. of NeurIPS, 2020.
- [25] J. Kaplan et al., "Scaling Laws for Neural Language Models." 2020.
- [26] J. Hoffmann et al., "Training Compute-Optimal Large Language Models." 2022.

- [27] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in Proc. of ICLR, 2020.
- [28] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, “Transformer Dissection: An Unified Understanding for Transformer’s Attention via the Lens of Kernel,” in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4344–4353. doi: 10.18653/v1/D19-1443.
- [29] A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, “The Impact of Positional Encoding on Length Generalization in Transformers,” in Thirty-seventh Conference on Neural Information Processing Systems, 2023. [Online]. Available: <https://openreview.net/forum?id=Drrl2gcjz1>